Sistema de Información y Control del Transmetro (SIC Transmetro)

Documento Unificado del Proyecto

Luis Orozco

Carné 3590 22 5339

Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas

Aseguramiento de la Calidad del Software

23 de octubre de 2026

2

Contenido y Enlaces del Proyecto

Este documento reúne en un solo archivo los entregables del Sistema de Información y Control del Transmetro (SIC Transmetro), proyecto final del curso de Aseguramiento de la Calidad del Software. Cada parte corresponde a un entregable que antes se presentaba por separado, actualizado a la versión del sistema del 23 de octubre de 2026.

Tabla 1

Contenido del documento

Parte	**Contenido**
Resumen ejecutivo	Qué es el sistema, qué resuelve y en qué estado se encuentra
Parte I	Justificación: para qué sirve, por qué se construyó así y qué problema resuelve
Parte II	Arquitectura, stack tecnológico y métricas de software
Parte III	Modelo entidad relación, script de base de datos y 15 diagramas UML
Parte IV	Documentación funcional: los doce módulos y la puesta en marcha
Parte V	Plan de pruebas y criterios de aceptación
Anexo A	Carta de aceptación de entrega
Referencias	Fuentes citadas en todo el documento

Nota. Elaboración propia.

Enlaces al Repositorio

El código fuente, los guiones de base de datos, los diagramas y este documento se encuentran publicados en el repositorio siguiente:

Tabla 2

Enlaces al repositorio del proyecto

Recurso	**Dirección**
Repositorio del proyecto	https://github.com/LuisO01/transmetro-sic
Guiones de base de datos	https://github.com/LuisO01/transmetro-sic/tree/main/01_database
Interfaz REST (Node.js)	https://github.com/LuisO01/transmetro-sic/tree/main/02_backend
Aplicación web (React)	https://github.com/LuisO01/transmetro-sic/tree/main/03_frontend
Entregables en PDF y diagramas	https://github.com/LuisO01/transmetro-sic/tree/main/entregables
Script consolidado de la base	https://github.com/LuisO01/transmetro-sic/blob/main/entregables/02_Script_BD_completo.sql
Fuentes editables de los diagramas	https://github.com/LuisO01/transmetro-sic/tree/main/entregables/fuentes_mermaid

Nota. El repositorio es público y no requiere credenciales. Las instrucciones de instalación y ejecución están en el archivo README.md de la raíz del repositorio y también en la Parte IV de este documento.

Resumen Ejecutivo

El SIC Transmetro centraliza en una sola base de datos la información de la red del Transmetro de la Ciudad de Guatemala y municipios aledaños —hoy dispersa entre municipalidades y operadores— y convierte en restricciones del motor de base de datos las reglas de operación que actualmente dependen de la verificación visual del personal.

La decisión de arquitectura que define al proyecto es dónde viven las reglas de negocio. No están en el código de la aplicación sino en PostgreSQL, de modo que ningún cliente puede transgredirlas: ni la aplicación web, ni un guion de carga, ni una consulta manual. Una unidad no puede quedar asignada a dos líneas, un espacio de parqueo no puede alojar dos unidades y una unidad con menos del 25 % de ocupación no puede despacharse antes de cinco minutos, porque el motor rechaza la operación.

Tabla 3

El sistema en cifras al 23 de octubre de 2026

Indicador	**Valor**
Requerimientos funcionales implementados	12 de 16, es decir 75 %
Módulos con interfaz	12
Pantallas de la aplicación web	15
Puntos de acceso de la interfaz REST	75
Tablas en la base de datos	33, con 12 vistas, 15 funciones y 29 disparadores
Restricciones de integridad	80, es decir 2.4 por tabla
Reglas de negocio garantizadas por el motor	7 de 7
Líneas de código	12,941 en 113 archivos
Casos de prueba ejecutados	38, todos con el resultado esperado
Defectos registrados	4, ninguno abierto

Nota. Cifras obtenidas por consulta al catálogo de PostgreSQL y por conteo sobre el repositorio.

Lo que queda pendiente está identificado y acotado: los módulos de administración técnica (RF-11), conciliación de pagos (RF-12) y reportería gerencial (RF-14) tienen sus tablas, reglas y vistas creadas en la base de datos, y resta construirles la interfaz. La deuda técnica principal no es funcional sino de verificación: los casos de prueba de los módulos de monitoreo y personal se ejecutaron de forma manual y su automatización es la primera tarea del sprint siguiente.

Parte I. Justificación del Sistema

El SIC Transmetro tiene como propósito centralizar en una sola base de datos la información de la red del Transmetro de la Ciudad de Guatemala y municipios aledaños, y automatizar las reglas de operación que actualmente dependen de la verificación manual. Esta parte expone, para el sistema en conjunto y para cada decisión de arquitectura, tres aspectos: para qué sirve, por qué se adoptó frente a las alternativas evaluadas y qué problema concreto resuelve.

Planteamiento del Problema

La información del Transmetro se encuentra distribuida entre cada municipalidad y cada operador, sin que exista una fuente oficial única. Las reglas de operación están documentadas en papel y se verifican mediante observación directa del personal.

Tabla 4

Situaciones identificadas en la operación actual y sus consecuencias

Situación actual	**Consecuencia**
Catálogos en archivos separados por municipalidad	El mismo dato existe en varias versiones y ninguna es oficial
Asignación de unidades registrada en hojas de cálculo	Una unidad puede quedar contabilizada en dos líneas o en ninguna
Control del rango de unidades por observación	La falta de unidades se detecta cuando el usuario ya espera en la estación
Parqueos asignados de forma verbal	Unidades que terminan la jornada sin espacio asignado
Cobertura de guardias en listados impresos	Accesos descubiertos que no se detectan oportunamente
Conteo de pasajeros sin registro sistemático	El sobrecupo no deja evidencia y no puede ser reclamado
Regla de espera de cinco minutos a criterio del operador	Se cumple de forma discrecional
Conciliación del recaudo con varios días de diferencia	Los faltantes se descubren cuando ya no es posible rastrearlos
Ausencia de registro de cambios	Ante un error no existe forma de determinar responsabilidad ni de reconstruir el hecho

Nota. Elaboración propia a partir del levantamiento de requerimientos del proyecto.

Propósito del Sistema

Tabla 5

Propósitos del sistema y usuarios destinatarios

Propósito	**Usuario destinatario**
Mantener un catálogo único y oficial de líneas, estaciones, accesos y unidades	Administrador municipal y administrador general
Conocer en todo momento cuántas unidades opera cada línea y si se encuentra dentro de su rango	Administrador municipal y administrador general
Garantizar que ninguna unidad esté asignada a dos líneas ni permanezca sin parqueo	Administrador municipal
Administrar el expediente de pilotos y la cobertura de guardias por acceso	Administrador municipal, piloto y guardia
Registrar la ocupación real de cada unidad y detectar el sobrecupo	Operador de estación
Hacer obligatoria la regla de espera de cinco minutos	Operador de estación
Conocer el estado del equipo de cómputo y la conectividad de cada estación	Operador de estación
Conciliar el recaudo por tarjeta el mismo día de la operación	Operador de estación
Obtener reportes consolidados de la red, exportables a formatos estándar	Administrador general
Determinar quién realizó cada cambio, cuándo y sobre qué valores	Auditoría y administración

Nota. Los usuarios corresponden a los cinco roles definidos en la matriz de permisos del sistema.

Fundamento de las Decisiones de Arquitectura

Cada decisión se expone bajo el mismo esquema: finalidad, fundamento frente a la alternativa descartada y problema que resuelve.

Base de Datos Relacional Centralizada

Finalidad. Disponer de un único lugar de consulta para cualquier dato de la red, con independencia de la municipalidad que lo haya capturado.

Fundamento. La alternativa evaluada consistía en mantener una base de datos por municipalidad y consolidarlas periódicamente. Ese esquema implica que, durante el intervalo entre consolidaciones, ninguna instancia posee el dato correcto, y que las diferencias deben resolverse manualmente. Se seleccionó PostgreSQL por tratarse de un motor de código abierto, requisito del proyecto, y por ofrecer las capacidades que el diseño requiere (PostgreSQL Global Development Group, s.f.).

Problema que resuelve. Elimina las versiones paralelas del mismo dato. Cada municipalidad conserva la propiedad de su información, representada mediante una columna en cada tabla, pero los datos residen en un solo repositorio.

Reglas de Negocio Implementadas en la Base de Datos

Finalidad. Asegurar que una regla se cumpla siempre, con independencia del medio por el cual ingresen los datos.

Fundamento. La práctica habitual consiste en programar las reglas en la capa de aplicación. Bajo ese esquema, la regla solo se cumple si el dato ingresó por esa vía: una carga masiva, un guion de corrección o una consulta manual la omiten sin generar aviso. Al implementarla en el motor, la validación forma parte del dato y no del programa que lo escribe. Esta decisión corresponde al principio de ubicar la lógica de integridad en la capa que la puede garantizar de forma incondicional (Sommerville, 2016).

Problema que resuelve. Que una regla especificada en el documento de requerimientos se cumpla efectivamente en producción y no únicamente en el flujo previsto de la aplicación.

Tabla 6

Reglas de negocio implementadas en el motor de base de datos

Requerimiento	Mecanismo	Situación que impide
RF-04. Una unidad en una sola línea activa	Restricción EXCLUDE USING gist sobre unidad y periodo	Que la misma unidad se contabilice en dos líneas, incluso ante asignaciones simultáneas
RF-06. Ninguna unidad sin parqueo	Dos restricciones EXCLUDE	Que dos unidades ocupen el mismo espacio o que una unidad figure en dos predios
RF-09. Sobrecupo	Columnas generadas con umbral del 150 % de la capacidad	Que la detección del sobrecupo dependa de un cálculo manual
RF-10. Espera de cinco minutos	Restricción CHECK sobre el registro del viaje	Que se despache con menos del 25 % de ocupación antes del tiempo mínimo
RF-05. Rango de unidades por línea	Disparador que abre y cierra la alerta	Que una línea opere fuera de su rango sin generar aviso
RF-08. Cobertura de guardias	Disparador que compara contra el mínimo del acceso	Que un acceso permanezca descubierto sin detección
RF-16. Bitácora de auditoría	Disparador genérico sobre 14 tablas	Que un cambio quede sin registro de autor y valor anterior

Nota. RF corresponde a requerimiento funcional. Las siete reglas fueron verificadas con datos inválidos sobre PostgreSQL 18.3.

Arquitectura en Tres Capas con Interfaz REST

Finalidad. Concentrar la lógica en un solo lugar y garantizar que todos los clientes, tanto la aplicación web actual como posibles aplicaciones móviles o tableros de análisis, obtengan resultados idénticos.

Fundamento. La alternativa era una aplicación que accediera directamente a la base de datos, lo cual obliga a repetir las validaciones en cada pantalla y a exponer el motor a cada equipo de la red, con el consiguiente riesgo de seguridad. Con una interfaz REST intermedia, la base acepta conexiones únicamente desde el servidor de aplicación.

Problema que resuelve. Evita que cada nueva pantalla reimplemente las reglas y permite modificar la interfaz sin alterar la lógica. Satisface además el requerimiento no funcional de capas desacopladas.

Interfaz Web Adaptable Basada en Componentes

Finalidad. Permitir el uso del sistema tanto desde la computadora de la estación como desde un dispositivo móvil en el andén.

Fundamento. El operador de estación y el guardia desempeñan sus funciones de pie. Una aplicación de escritorio los obligaría a regresar a la caseta para registrar lo observado, y el registro diferido pierde exactitud. Se seleccionó React por su modelo de componentes reutilizables y por la disponibilidad de documentación y soporte, factor relevante en un sistema que será mantenido por terceros.

Problema que resuelve. Que la ocupación de una unidad se registre en el momento y en el lugar en que ocurre. El módulo de monitoreo, construido en octubre de 2026, materializa esta decisión: el operador registra la llegada, la ocupación y el despacho desde el andén.

Matriz de Permisos Almacenada como Dato

Finalidad. Que cada uno de los cinco roles acceda y modifique exactamente lo que le corresponde.

Fundamento. Programar los permisos mediante condiciones en el código implica que cada ajuste requiere modificación, prueba y despliegue del programa. Al almacenarlos en una tabla, el ajuste consiste en actualizar un registro.

Problema que resuelve. Que un piloto no acceda al expediente de otro, que un operador no modifique el catálogo de líneas y que un administrador municipal no altere datos de otra municipalidad. Adicionalmente, corregir un permiso mal asignado toma minutos y no un ciclo de despliegue.

Ámbito de Datos Independiente del Nivel de Permiso

Finalidad. Distinguir qué operaciones puede ejecutar un usuario de cuáles registros puede afectar.

Fundamento. Dos administradores municipales poseen el mismo nivel de permiso pero no deben acceder a los mismos datos. Combinar ambos conceptos obligaría a crear un rol por municipalidad, con un crecimiento incontrolado del catálogo de roles.

Problema que resuelve. Que el sistema escale a nuevos municipios sin rediseño: incorporar uno consiste en insertar un registro y no en crear roles adicionales.

Alertas Generadas por el Motor

Finalidad. Detectar las situaciones anómalas en el instante en que se producen.

Fundamento. La alternativa consistía en un proceso de revisión periódica, que deja una ventana sin cobertura entre ejecuciones y consume recursos consultando información que no ha cambiado. Un disparador reacciona al cambio mismo.

Problema que resuelve. Que la alerta por una línea bajo su mínimo exista en el mismo instante en que una unidad ingresa a mantenimiento, y que se cierre automáticamente al normalizarse la condición.

Datos Derivados no Almacenados

Finalidad. Que la longitud de una línea, el porcentaje de ocupación y la diferencia de conciliación coincidan siempre con los datos que los originan.

Fundamento. Un total almacenado requiere actualización explícita. La omisión de esa actualización produce información incorrecta presentada con apariencia de validez.

Problema que resuelve. La existencia de reportes contradictorios entre sí.

Bitácora de Auditoría con Identificación del Usuario Real

Finalidad. Permitir la reconstrucción de qué ocurrió, cuándo y por decisión de quién.

Fundamento. Lo habitual es que la aplicación registre en la bitácora de forma discrecional y que el motor identifique al usuario técnico de la conexión, común a todos. En este diseño, el identificador de la persona se publica en la sesión de base de datos en cada transacción, y un disparador genérico cubre 14 tablas críticas.

Problema que resuelve. Que ante un dato incorrecto se determine quién lo modificó y cuál era su valor anterior. La información del sistema está sujeta al régimen de acceso a la información pública (Congreso de la República de Guatemala, 2008), por lo que la trazabilidad constituye una obligación legal y no una característica opcional.

Parámetros del Negocio Almacenados como Dato

Finalidad. Que los umbrales de la operación —la espera mínima, la ocupación mínima, el umbral de sobrecupo, las tarifas y la ventana de transbordo— puedan ajustarse sin tocar el código.

Fundamento. Un umbral escrito en el código obliga a un ciclo completo de modificación, prueba y despliegue cada vez que la autoridad del servicio lo cambia. En una tabla, el cambio es una actualización con constancia de quién la hizo y cuándo.

Problema que resuelve. Que una decisión administrativa, como modificar la tarifa del servicio exprés, dependa del calendario del equipo de desarrollo.

Base de Datos Administrada en la Nube

Finalidad. Contar con respaldo automático, réplica y procedimiento de recuperación sin un equipo dedicado de infraestructura.

Fundamento. Un servidor propio exige verificación diaria de los respaldos, actividad que en la práctica se omite hasta que surge la necesidad de restaurar.

Problema que resuelve. Los requerimientos no funcionales de respaldo automático, plan de recuperación ante desastres y disponibilidad igual o mayor al 99 % en horario operativo.

Problemática Resuelta por Requerimiento

Tabla 7

Aporte de cada requerimiento funcional

Requerimiento	Finalidad	Problema manual que elimina	Estado
RF-01	Catálogo de líneas y estaciones	Listados paralelos por municipalidad	Implementado
RF-02	Distancia entre estaciones y longitud de línea	Cálculo manual y totales desactualizados	Implementado
RF-03	Inventario de accesos	Desconocimiento de la cantidad de guardias requeridos	Implementado
RF-04	Una unidad en una sola línea activa	Doble asignación y conteo inflado de la flota	Implementado
RF-05	Alerta por rango de unidades	Detección tardía de la falta de servicio	Implementado
RF-06	Asignación de parqueos	Unidades sin espacio asignado al final de la jornada	Implementado
RF-07	Expediente de pilotos	Expedientes en papel y licencias vencidas sin detectar	Implementado
RF-08	Cobertura de guardias por acceso	Accesos descubiertos detectados fuera de tiempo	Implementado
RF-09	Ocupación y sobrecupo	Ausencia de evidencia del sobrecupo	Implementado
RF-10	Regla de espera de cinco minutos	Cumplimiento discrecional de la regla	Implementado
RF-11	Estado de equipo y conectividad	Mantenimiento reactivo en lugar de planificado	Pendiente de interfaz
RF-12	Conciliación de pagos	Descuadres detectados cuando ya no son rastreables	Pendiente de interfaz
RF-13	Asociación a municipalidad	Indefinición de responsabilidades por estación y línea	Implementado
RF-14	Reportes consolidados	Recolección manual de datos ante cada operador	Pendiente
RF-15	Usuarios y roles	Cuentas compartidas y accesos indebidos	Implementado
RF-16	Bitácora de auditoría	Imposibilidad de rastrear cambios ante reclamos o auditorías	Implementado en la base

Nota. Elaboración propia a partir de la especificación de requerimientos del proyecto. "Pendiente de interfaz" significa que las tablas, las reglas y las vistas existen y fueron verificadas, pero el módulo de pantalla no se ha construido.

Conclusión de la Justificación

El sistema convierte reglas que actualmente dependen de la verificación humana en restricciones que el motor de base de datos no permite transgredir, y conserva evidencia de cada cambio. Las nueve situaciones problemáticas identificadas en la operación actual quedan cubiertas por los dieciséis requerimientos funcionales, de los cuales doce se encuentran implementados y verificados.

La decisión de mayor impacto es la ubicación de las reglas de negocio en el motor de base de datos, y su efecto es verificable: durante las pruebas, los intentos de asignar una unidad a dos líneas, de ocupar un espacio de parqueo ya asignado y de despachar una unidad antes del tiempo mínimo fueron rechazados por el motor, sin intervención del código de la aplicación. Al construir el módulo de monitoreo quedó demostrada además la segunda virtud de esa decisión: las reglas ya estaban probadas, de modo que la pantalla se limitó a presentarlas.

Parte II. Arquitectura, Stack Tecnológico y Métricas de Software

El Sistema de Información y Control del Transmetro (SIC Transmetro) centraliza la información de las líneas, estaciones y unidades del sistema de transporte de la Ciudad de Guatemala y municipios aledaños. El presente documento desarrolla dos aspectos del proyecto: el stack tecnológico, entendido como el conjunto de lenguajes, bibliotecas y servicios que el sistema emplea en cada capa, y el plan de métricas de software, que define qué se mide, cómo se mide y con qué propósito.

Esta es la segunda medición del proyecto. La primera se realizó el 2 de octubre de 2026, al cierre del Sprint 0. Entre ambas fechas se construyeron cuatro módulos adicionales con interfaz —monitoreo de capacidad y servicio, personal, mi expediente y mi cobertura— y se reconstruyó el tablero con mapa de la red e indicadores gráficos. Todos los valores que se presentan corresponden a la medición del 19 de octubre de 2026 y, cuando resulta pertinente, se contrastan con los de la medición anterior.

El Stack Tecnológico

El sistema adopta una arquitectura en tres capas desacopladas. La característica distintiva de su stack es el uso de un solo lenguaje de programación de aplicación, JavaScript, tanto en el navegador como en el servidor, con SQL y PL/pgSQL reservados para la capa de datos. Todos los componentes son de código abierto y carecen de costo de licencia.

La capa de presentación se comunica con la capa de aplicación mediante HTTPS en el puerto 443 con formato JSON, y la capa de aplicación se comunica con la capa de datos mediante el protocolo de PostgreSQL en el puerto 5432 con cifrado TLS.

Conviene señalar un resultado de la medición antes de presentar la tabla: la construcción de los cuatro módulos nuevos, del mapa de la red y de las tres gráficas no incorporó ninguna dependencia adicional. El stack es exactamente el mismo que el del Sprint 0, con las mismas versiones. El mapa y las gráficas se dibujan en SVG escrito a mano, decisión tomada porque una biblioteca de gráficas pesaría más que todo el código de la interfaz y porque un mapa que

depende de un servicio externo deja de ser útil precisamente en la estación con conectividad intermitente.

Tabla 8

Stack tecnológico completo del sistema por capa

Capa	Componente	Versión	Función en el sistema
Presentación	React y React DOM	18.3.1	Construcción de la interfaz mediante componentes reutilizables
Presentación	React Router DOM	6.28.0	Enrutamiento de las quince pantallas y protección de rutas por módulo y nivel
Presentación	Vite	6.0.7	Servidor de desarrollo con recarga inmediata y empaquetado para producción
Presentación	Plugin de React para Vite	4.3.4	Transformación de la sintaxis JSX a JavaScript
Presentación	JavaScript con JSX	ECMAScript 2022	Lenguaje de la interfaz
Presentación	CSS3 propio	Estándar del W3C	Hoja de estilos de 556 líneas, adaptable desde 820 píxeles
Presentación	SVG propio	Estándar del W3C	Mapa de la red y tres gráficas, sin biblioteca de terceros
Aplicación	Node.js	20 LTS o superior	Entorno de ejecución de JavaScript en el servidor
Aplicación	Express	4.21.2	Marco de trabajo de la interfaz REST y encadenamiento de middlewares
Aplicación	JavaScript con módulos ES	ECMAScript 2022	Lenguaje del servidor
Aplicación	pg	8.13.1	Controlador de PostgreSQL con agrupación de conexiones
Aplicación	zod	3.24.1	Validación y conversión de tipos de los datos recibidos
Aplicación	jsonwebtoken	9.0.2	Emisión y verificación de los tokens de acceso y de refresco
Aplicación	bcryptjs	3.0.2	Cifrado de contraseñas con factor de coste 12
Aplicación	Helmet	8.0.0	Cabeceras de seguridad HTTP
Aplicación	CORS	2.8.5	Control de orígenes permitidos
Aplicación	express-rate-limit	7.5.0	Límite de peticiones por dirección, con restricción adicional en el inicio de sesión
Aplicación	morgan	1.10.0	Registro de peticiones

Aplicación	dotenv	16.4.5	Carga de variables de entorno
Datos	PostgreSQL	14 o superior, verificado en 18.3	Motor relacional; almacena los datos y garantiza las reglas de negocio
Datos	SQL	SQL:2016, dialecto PostgreSQL	Definición del esquema, consultas y vistas
Datos	PL/pgSQL	Nativo del motor	15 funciones y 29 disparadores de reglas de negocio y auditoría
Datos	Extensión btree_gist	Del motor	Habilita las restricciones de exclusión que combinan igualdad con rangos de tiempo (RF-04 y RF-06)
Datos	Extensión citext	Del motor	Comparación de usuario y correo sin distinguir mayúsculas
Datos	Extensión pgcrypto	Del motor	Funciones criptográficas; el cálculo de hash se realiza actualmente en la capa de aplicación
Calidad	Vitest	2.1.9	Ejecución de pruebas unitarias y de integración
Calidad	Supertest	7.0.0	Pruebas de los puntos de acceso REST sin levantar el servidor
Calidad	PGlite	0.5.8	Ejecución de PostgreSQL dentro de las pruebas y como servidor local, sin instalar un motor
Calidad	ESLint	9.17.0	Análisis estático del código
Calidad	Mermaid CLI	11	Generación de los diagramas a partir de fuentes de texto versionadas
Calidad	Git	2.51	Control de versiones del código, del guion y de los diagramas
Calidad	npm	10 o superior	Gestión de dependencias y ejecución de guiones
Despliegue	Hosting estático o red de distribución de contenido	Servicio en la nube	Entrega de los archivos generados por Vite
Despliegue	Balanceador de carga	Servicio con certificado TLS	Terminación de HTTPS y distribución entre instancias
Despliegue	Servidor de aplicación	Node.js 20 LTS	Ejecución de la interfaz REST; sin estado en memoria, admite varias instancias
Despliegue	Base de datos administrada	PostgreSQL	Respaldo diario, registro continuo de transacciones y réplica

Nota. Las capas de presentación, aplicación y datos constituyen el producto desplegado. Las herramientas de calidad determinan cómo se construye y se verifica el sistema, pero no se despliegan. El esquema contiene 33 tablas, 12 vistas, 77 llaves foráneas y 80 restricciones de integridad. No se emplea biblioteca de componentes, framework de CSS ni biblioteca de gráficas de terceros.

Tabla 9

Requisitos del entorno de ejecución

Requisito	**Versión mínima**	**Ámbito**
Node.js	20	Desarrollo y servidor de producción
npm	10	Desarrollo
PostgreSQL	14	Base de datos, local o administrada
Navegador web	Versión vigente de Chrome, Edge, Firefox o Safari	Equipos de estación y dispositivos móviles
Git	2.40	Desarrollo

Nota. Los equipos de estación no requieren instalación de software adicional, puesto que la aplicación se ejecuta en el navegador. La ausencia de estado en memoria en el servidor de aplicación es la condición que permite escalar de forma horizontal.

Las Métricas de Software

Una métrica de software es una medida cuantitativa de un atributo de un producto, de un proceso o de un proyecto de software. Su propósito no es describir, sino permitir decisiones: establecer si un entregable cumple los criterios de aceptación, detectar desviaciones de forma temprana y sustentar las estimaciones de esfuerzo (Pressman y Maxim, 2020).

Este proyecto clasifica sus métricas en tres grupos, conforme a lo que cada una mide.

Tabla 10

Clasificación de las métricas del proyecto

Grupo	Qué mide	Ejemplo en este proyecto
Métricas de producto	Atributos del software construido	Tamaño del código, cantidad de restricciones en la base de datos, tiempo de respuesta
Métricas de proceso	Atributos de la forma en que se construye	Defectos detectados por fase, cumplimiento de la definición de terminado
Métricas de proyecto	Atributos del avance y del esfuerzo	Requerimientos implementados por sprint, velocidad del equipo

Nota. Elaboración propia con base en Pressman y Maxim (2020).

Las características de calidad evaluadas se alinean con el modelo de calidad del producto de la norma ISO/IEC 25010 (ISO/IEC, 2011), en particular las de adecuación funcional, eficiencia de desempeño, fiabilidad, seguridad y mantenibilidad.

Métricas de Producto: Tamaño

Las mediciones se realizaron el 19 de octubre de 2026 sobre el repositorio, excluyendo dependencias externas y archivos generados. El conteo de líneas incluye comentarios y líneas en blanco.

Tabla 11

Tamaño del código fuente por lenguaje

Lenguaje	Archivos	Líneas	Participación
JavaScript (servidor y utilidades)	47	4,737	37 %
JavaScript con JSX (interfaz)	24	3,494	27 %
SQL y PL/pgSQL	12	2,423	19 %
Mermaid (diagramas)	24	1,641	13 %
CSS	2	557	4 %
HTML y JSON	4	89	menos de 1 %
Total	113	12,941	100 %

Nota. Total de 12,941 líneas en 113 archivos, frente a 10,040 líneas en 96 archivos en la medición del 2 de octubre de 2026: un incremento del 29 %. Los diagramas en Mermaid se contabilizan como código porque se versionan y se revisan igual que el resto del repositorio. La documentación en formato Markdown suma 1,136 líneas adicionales, no incluidas en el total.

Tabla 12

Tamaño del modelo de datos y de la interfaz de programación

Elemento	**2 de octubre**	**19 de octubre**
Tablas	32	33
Vistas	12	12
Tipos enumerados	31	31
Funciones en PL/pgSQL	15	15
Disparadores	29	29
Índices creados explícitamente	37	37
Llaves foráneas	76	77
Restricciones CHECK	47	49
Restricciones UNIQUE	28	28
Restricciones EXCLUDE	3	3
Columnas generadas	4	4
Módulos del servidor	9	12
Puntos de acceso REST	51	75
Pantallas de la aplicación web	11	15

Nota. Valores obtenidos por consulta al catálogo del sistema tras ejecutar el guion completo, y por conteo sobre el código fuente. La tabla nueva es parametro_sistema, que externaliza los umbrales del negocio —espera mínima, ocupación mínima, umbral de sobrecupo, tarifas y ventana de transbordo— de modo que cambiarlos no exija modificar el código. Las dos restricciones CHECK nuevas y la llave foránea nueva pertenecen a esa tabla y a los campos de tarifa, horario y frecuencia agregados a la tabla de líneas. La medición anterior reportó cuatro

restricciones EXCLUDE; la verificación contra el catálogo del motor confirma que son tres, y el valor se corrige en esta edición.

La relación entre restricciones declarativas y tamaño del esquema constituye en sí misma un indicador: 80 restricciones de integridad para 33 tablas, es decir, 2.4 restricciones por tabla. Cuanto mayor es esa proporción, menor es la cantidad de validaciones que deben programarse y mantenerse en la capa de aplicación.

Métricas de Producto: Calidad

Tabla 13

Fichas de las métricas de producto

Métrica	Fórmula	Unidad	Meta	Instrumento	Frecuencia
Cobertura de pruebas en servicios	Líneas ejecutadas por las pruebas entre líneas totales de los módulos, por cien	Porcentaje	Igual o mayor a 70 %	Vitest con reporte de cobertura	Al cierre de cada sprint
Cobertura de requerimientos	Requerimientos con al menos un caso de prueba automatizado entre requerimientos totales, por cien	Porcentaje	100 % de los requerimientos con regla de negocio	Matriz de trazabilidad	Al cierre de cada sprint
Densidad de defectos	Defectos encontrados entre miles de líneas de código	Defectos por millar	Menos de 5	Registro de defectos	Al cierre de cada sprint
Defectos escapados	Defectos hallados después de la entrega entre defectos totales, por cien	Porcentaje	Menos de 10 %	Registro de defectos	Al final del proyecto
Reglas garantizadas en la base de datos	Reglas con restricción o disparador entre reglas de negocio totales, por cien	Porcentaje	100 % de las reglas duras	Revisión del guion	Al cierre de cada sprint
Tiempo de respuesta	Percentil 95 del tiempo de los diez puntos de acceso más utilizados	Segundos	Menos de 3	Prueba de carga con k6	Sprint 7 y previo a la entrega
Disponibilidad	Minutos en servicio entre minutos del horario operativo, por cien	Porcentaje	Igual o mayor a 99 %	Sonda automática al punto de acceso de salud	Mensual

Nota. Las metas de tiempo de respuesta y disponibilidad provienen de los requerimientos no funcionales del proyecto. El término regla dura designa aquellas cuya violación produce datos

inconsistentes.

Métricas de Proceso y de Proyecto

Tabla 14

Fichas de las métricas de proceso y de proyecto

Métrica	**Fórmula**	**Propósito**
Velocidad del equipo	Historias de usuario terminadas por sprint	Proyectar si el alcance restante cabe en el tiempo disponible
Cumplimiento de la definición de terminado	Historias que cumplen los siete criterios entre historias cerradas, por cien	Detectar avance aparente que en realidad acumula deuda técnica
Defectos por fase de detección	Conteo de defectos hallados en diseño, en pruebas y en producción	Sustentar que el costo de corrección se incrementa conforme el defecto avanza de fase
Retrabajo	Horas dedicadas a corrección entre horas totales, por cien	Cuantificar el costo de la mala calidad
Tiempo medio de corrección	Promedio de días entre el reporte de un defecto y su cierre	Evaluar la capacidad de respuesta del equipo
Avance de requerimientos	Requerimientos implementados entre requerimientos totales, por cien	Comunicar el progreso real del proyecto

Nota. Elaboración propia con base en Pressman y Maxim (2020) y Sommerville (2016).

Valores Medidos al 19 de Octubre de 2026

Tabla 15

Estado actual de las métricas

Métrica	Valor anterior	Valor actual	Meta	Estado
Pruebas automatizadas	9, todas satisfactorias	9, todas satisfactorias	Incremento por sprint	No cumplido
Casos de prueba ejecutados	26	38	Incremento por sprint	Cumplido
Cobertura de pruebas en servicios	Sin medir	Sin medir	Igual o mayor a 70 %	Pendiente
Reglas de negocio garantizadas en la base	7 de 7 reglas duras	7 de 7 reglas duras	100 %	Cumplido
Reglas verificadas con datos inválidos	9 pruebas ejecutadas	9 pruebas ejecutadas	100 % de las reglas duras	Cumplido
Restricciones de integridad por tabla	2.5	2.4	Sin meta definida; se reporta como indicador	Informativo
Avance de requerimientos	7 de 16, es decir 44 %	12 de 16, es decir 75 %	100 %	En curso
Trazabilidad de requerimientos	16 de 16 documentados	16 de 16 documentados	100 %	Cumplido
Densidad de defectos	2 defectos en 10,040 líneas, 0.20 por millar	4 defectos en 12,941 líneas, 0.31 por millar	Menos de 5 por millar	Cumplido
Tiempo medio de corrección	Menos de un día	Menos de un día en los cuatro defectos	Sin meta definida aún	Informativo
Tiempo de respuesta	Sin medir	Sin medir	Menos de 3 segundos	Pendiente

Nota. Las reglas duras corresponden a los requerimientos funcionales RF-04, RF-05, RF-06, RF-08, RF-09, RF-10 y RF-16. Los cuatro defectos registrados figuran en el plan de pruebas como DEF-01 a DEF-04; los dos últimos, de severidad crítica y alta, se detectaron al conectar el sistema a una base de datos real por primera vez.

El dato que exige atención no es ninguno de los que cumplen su meta, sino el que no la cumple: la cantidad de pruebas automatizadas permanece en nueve mientras el código creció un 29 %. Los cuatro módulos nuevos se verificaron de forma manual, con los casos CP-27 a CP-38, y esa verificación manual no se repite sola en cada cambio. La consecuencia práctica es que una modificación en el módulo de monitoreo puede romper la regla de los cinco minutos sin que ninguna prueba lo advierta. La acción correctiva correspondiente, automatizar los casos CP-27 a CP-38 con Vitest y Supertest, queda registrada como la primera tarea del sprint siguiente.

Umbrales y Acciones Correctivas

Una métrica sin una acción asociada no modifica el comportamiento del proyecto. La tabla siguiente define qué hacer cuando un valor se aparta de su meta.

Tabla 16

Umbrales de alerta y acciones correctivas

Métrica	**Umbral de alerta**	**Acción correctiva**
Cobertura de pruebas	Menor a 70 % al cierre del sprint	No se cierra el sprint hasta agregar las pruebas faltantes de los servicios modificados
Pruebas automatizadas	Un módulo nuevo sin prueba automatizada de su regla de negocio	Se automatizan sus casos antes de iniciar el módulo siguiente
Cobertura de requerimientos	Un requerimiento con regla de negocio sin caso de prueba	Se agrega el caso antes de dar la historia por terminada
Densidad de defectos	Mayor a 5 por millar de líneas	Revisión por pares obligatoria del módulo afectado en el sprint siguiente
Reglas garantizadas en la base	Menor a 100 %	La historia no cumple la definición de terminado y se reabre
Tiempo de respuesta	Mayor a 3 segundos en cualquier punto de acceso	Análisis del plan de ejecución de la consulta y revisión de índices
Disponibilidad	Menor a 99 % en el mes	Análisis de causa raíz documentado y plan de corrección
Defectos en producción	Cualquier defecto escapado	Se agrega un caso de prueba que lo reproduzca antes de corregirlo

Nota. Elaboración propia. El umbral de pruebas automatizadas se incorpora en esta edición como consecuencia directa de la medición del 19 de octubre de 2026.

Procedimiento de Recolección

Las métricas automatizables se obtienen mediante los guiones siguientes, ejecutados desde la carpeta del servidor:

```
npm test                    Ejecuta las pruebas automatizadas
npx vitest run --coverage   Genera el reporte de cobertura de lineas y
ramas
npm run db:local:reset      Recrea el esquema y verifica el guion
completo
```

Las métricas de tamaño se obtienen por conteo sobre el repositorio y por consulta al catálogo del sistema de PostgreSQL. Las métricas de proceso se registran de forma manual en la bitácora del proyecto al cierre de cada sprint. La verificación del guion contra un motor real se realiza con PGlite, lo cual permite ejecutar las pruebas de reglas de negocio en cualquier equipo sin instalar un servidor de base de datos.

Conclusiones sobre el Stack y las Métricas

El stack del SIC Transmetro se compone de siete tecnologías en la capa de presentación, once en la capa de aplicación y seis en la capa de datos, además de siete herramientas de desarrollo y aseguramiento de la calidad. Todas son de código abierto. La decisión de mayor peso sigue correspondiendo al motor de base de datos: PostgreSQL se seleccionó por una capacidad concreta, las restricciones de exclusión sobre rangos temporales, que garantizan los requerimientos RF-04 y RF-06 sin depender del código de la aplicación. La construcción de los cuatro módulos nuevos confirmó la segunda virtud de esa decisión: las reglas de los requerimientos RF-09 y RF-10 ya estaban en el motor, de modo que la interfaz de monitoreo se limitó a presentarlas.

El plan de métricas comprende siete métricas de producto, seis de proceso y de proyecto, y un conjunto de umbrales con su acción correctiva asociada. Entre las dos mediciones, el avance de requerimientos pasó del 44 % al 75 %, el código creció un 29 % y la densidad de defectos se

mantuvo muy por debajo de su umbral. En contrapartida, la cantidad de pruebas automatizadas no creció, lo cual constituye la deuda técnica más relevante del proyecto en este momento y la prioridad del sprint siguiente.

Parte III. Modelo Entidad Relación, Script de Base de Datos y Diagramas UML

Alcance de esta Parte

Este documento reúne el diseño de datos y el diseño UML del Sistema de Información y Control del Transmetro (SIC Transmetro): el diagrama entidad relación, el script de la base de datos y 15 diagramas UML. Es el entregable de las fases F6 a F8 del proyecto final del curso de Aseguramiento de la Calidad del Software.

El sistema centraliza la información de las 7 líneas, 106 estaciones y 210 unidades que constituyen la red objetivo del Transmetro de la Ciudad de Guatemala y municipios aledaños, y automatiza en la base de datos las reglas de negocio que hoy dependen del control manual. Los datos de ejemplo cargan una porción representativa de esa red: 7 líneas, 26 estaciones y 170 unidades.

Esta es la segunda edición del documento. Reemplaza a la del 2 de octubre de 2026 e incorpora los cambios al modelo introducidos al construir los módulos de monitoreo, personal y tablero: la tabla de parámetros del sistema y los campos de tarifa, horario y frecuencia de la tabla de líneas.

Tabla 17

Tecnologías utilizadas por capa del sistema

Capa	Tecnología
Frontend	React 18 y Vite, aplicación web adaptable
Backend	Interfaz REST en Node.js con Express
Base de datos	PostgreSQL 14 o superior, servicio administrado en la nube
Seguridad	Token por usuario y rol, contraseñas con bcrypt, HTTPS

Nota. El script se ejecutó completo en PostgreSQL 18.3 mediante PGlite 0.5.8 el 19 de octubre de 2026. Creó las 33 tablas y las 12 vistas sin errores. Las pruebas confirmaron que las reglas RF-04, RF-06 y RF-10 rechazan los datos inválidos, y que RF-05, RF-08, RF-09 y RF-16

generan su alerta o su registro en la bitácora. Los diagramas se generaron con Mermaid a partir de fuentes de texto versionadas, que se entregan en la carpeta entregables\fuentes_mermaid.

Diagrama Entidad Relación

El modelo tiene 33 tablas en 6 dominios, todas en el esquema transmetro. Primero se presenta el modelo completo y después un diagrama por dominio, que es el que se lee en tamaño carta.

Notación. Pata de gallo. Dos barras indican exactamente uno; barra y círculo, cero o uno; círculo y llave, cero o muchos. PK es llave primaria, FK llave foránea y UK llave única. En los diagramas por dominio, una entidad que aparece solo con su identificador y la nota "ver su dominio" pertenece a otro dominio.

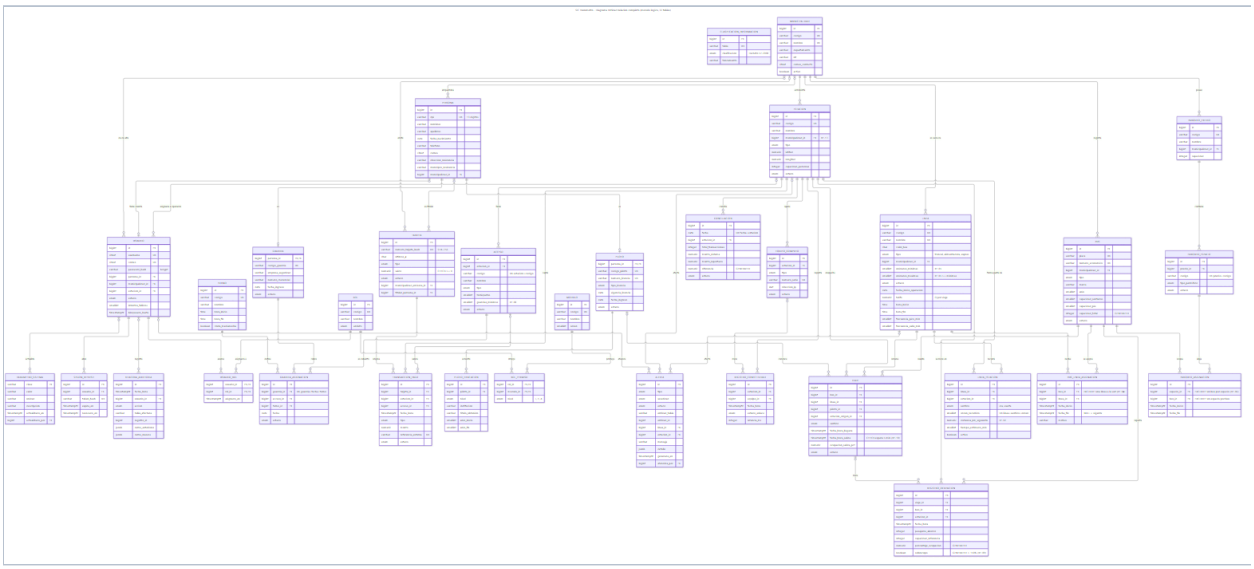
Convenciones del modelo:

- Toda tabla tiene una llave sustituta id y una llave natural única: codigo, placa o dpi.
- Los borrados son lógicos, mediante estado o activo, porque la bitácora exige trazabilidad permanente.
- Los datos derivados no se guardan: la longitud de una línea se calcula con la vista v_linea_longitud.
- Las asignaciones con historia, bus a línea y bus a parqueo, tienen vigencia: fecha_fin en nulo significa asignación vigente.
- Los umbrales del negocio no están escritos en el código: residen en la tabla parametro_sistema.

Modelo Completo

Figura 1

Diagrama entidad relación completo, 33 tablas

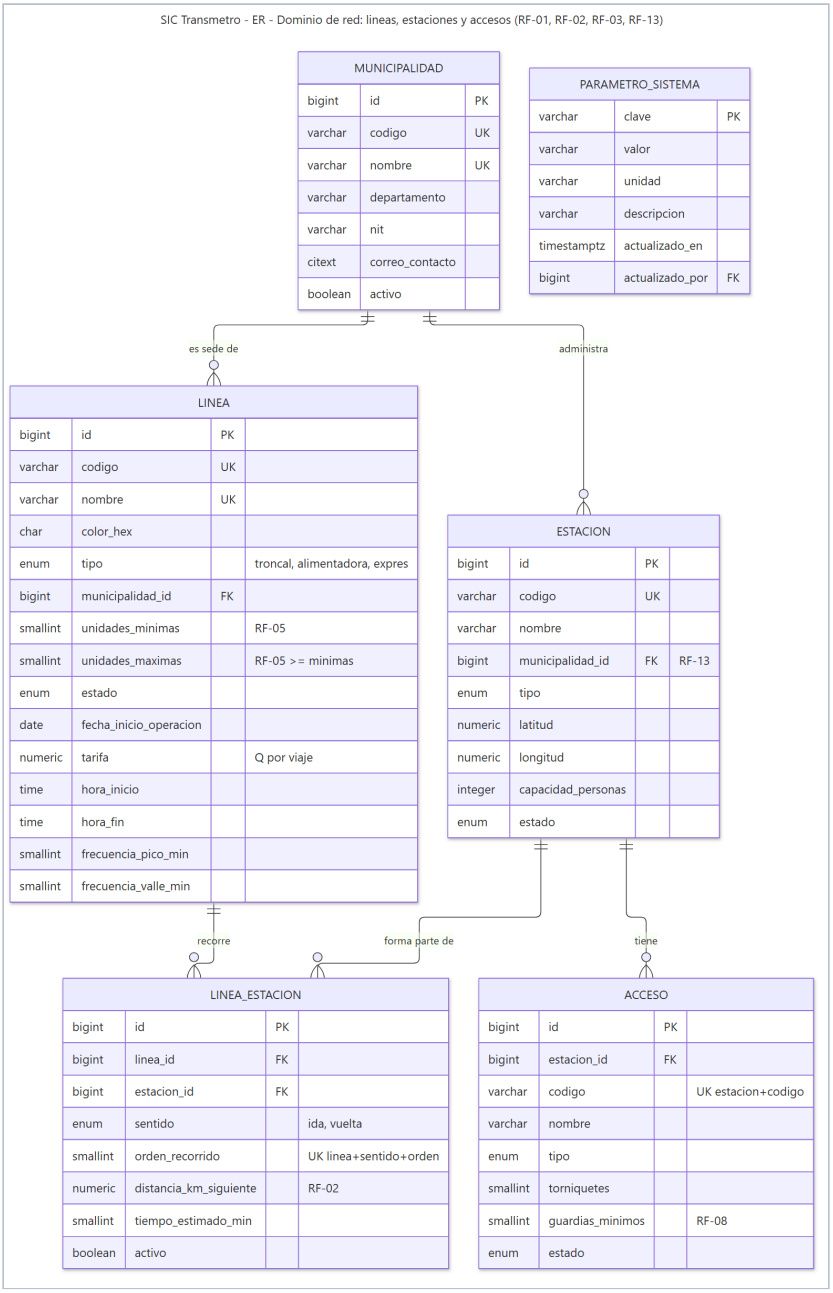


Nota. La tabla parametro_sistema es la incorporación de esta edición. Guarda la espera mínima, la ocupación mínima, el umbral de sobrecupo, las tarifas regular y de servicio exprés, el costo de la tarjeta y la ventana de transbordo, con su unidad y la marca de quién los actualizó.

Dominio de Red: Líneas, Estaciones y Accesos (RF-01, RF-02, RF-03, RF-13)

Figura 2

ER del dominio de red



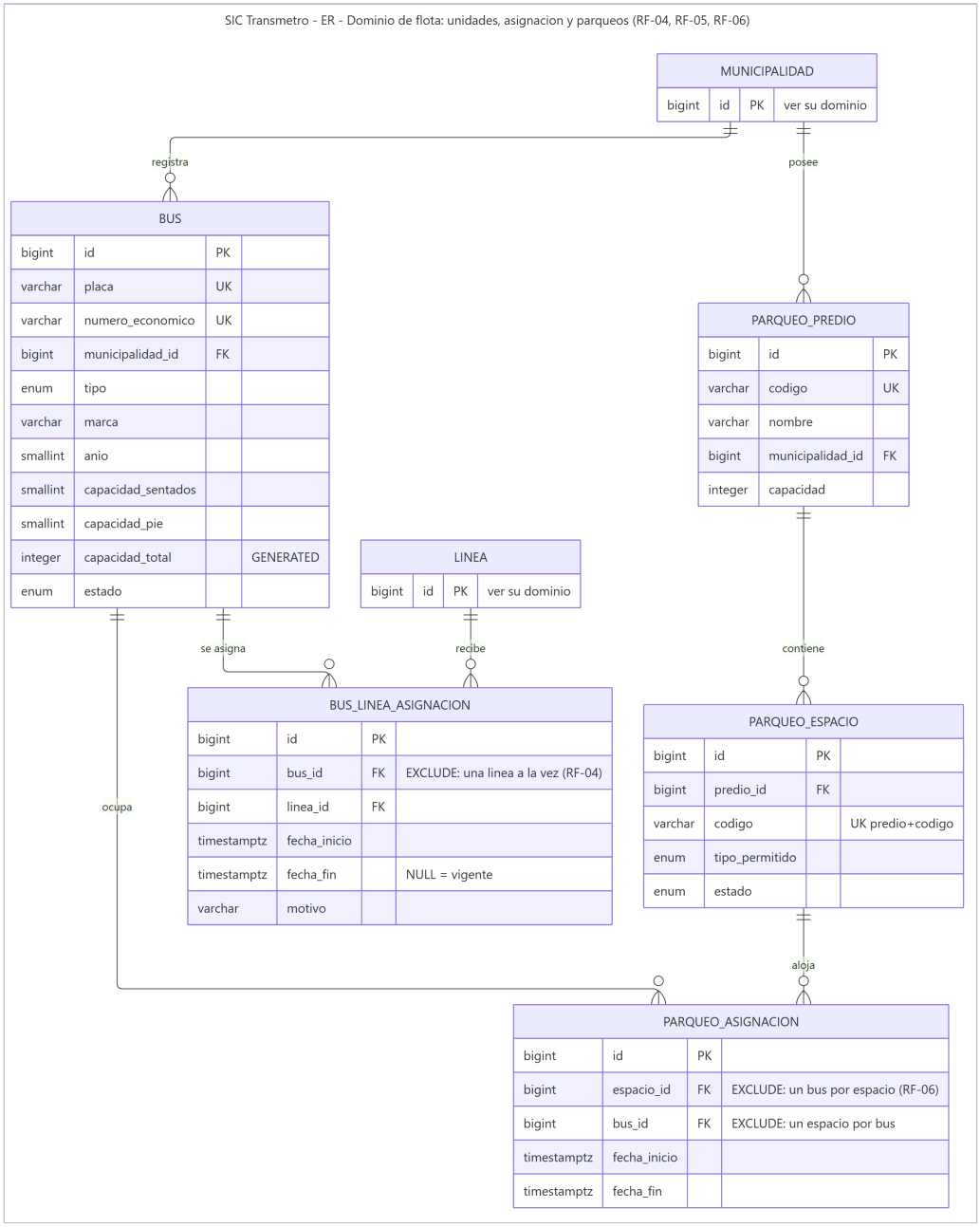
Nota. LINEA y ESTACION tienen una relación muchos a muchos que se resuelve con LINEA_ESTACION, la cual guarda el sentido, el orden de recorrido y la distancia a la siguiente

estación. El orden no se repite dentro de la misma línea y sentido. En esta edición LINEA incorpora su tarifa, su horario de operación y sus frecuencias de hora pico y hora valle, con restricciones de verificación que impiden una tarifa negativa, un horario invertido o una frecuencia valle menor que la de hora pico.

Dominio de Flota: Unidades, Asignación y Parqueos (RF-04, RF-05, RF-06)

Figura 3

ER del dominio de flota



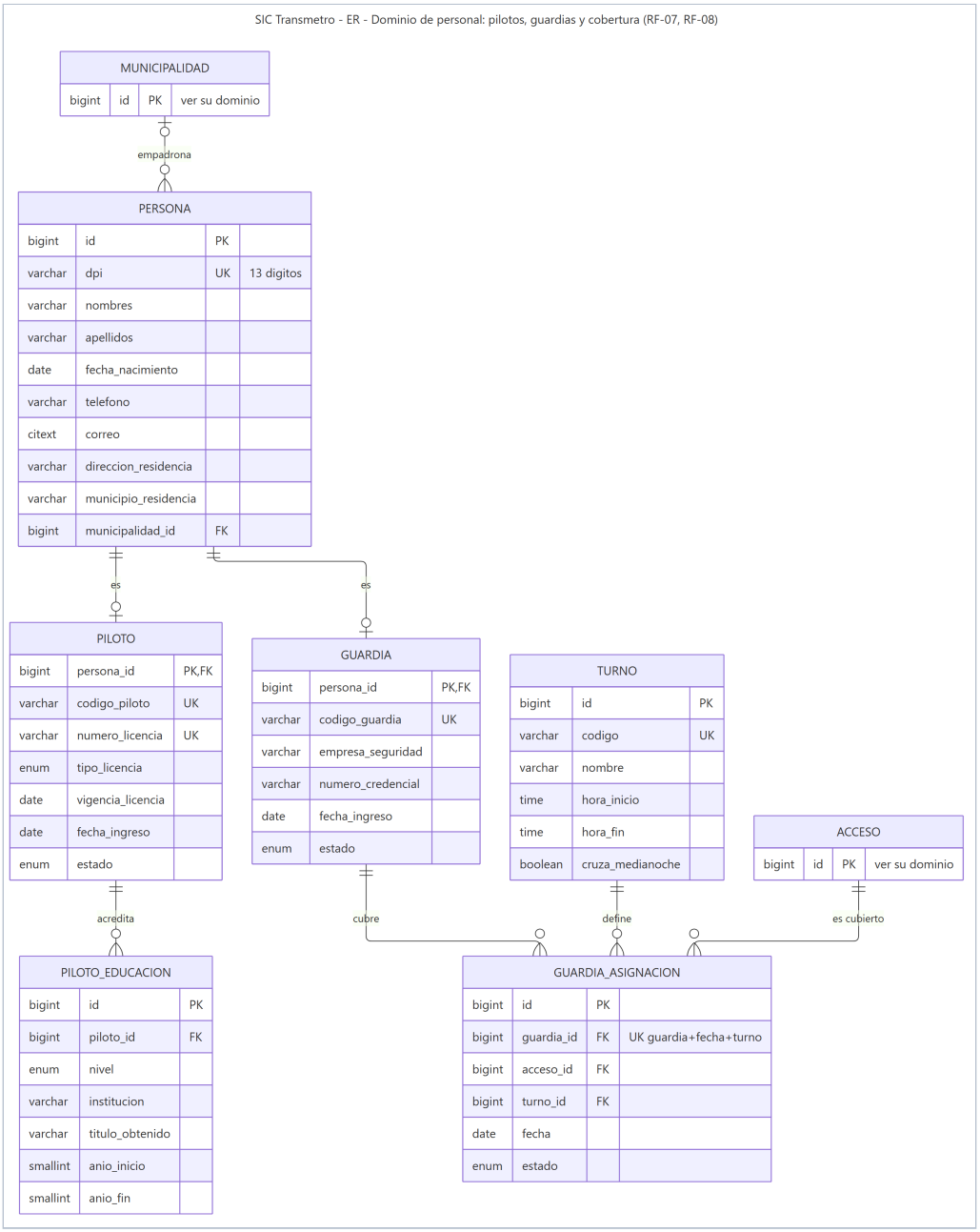
Nota. BUS_LINEA_ASIGNACION y PARQUEO_ASIGNACION guardan la historia completa. Las restricciones de exclusión impiden que una unidad esté en dos líneas o en dos

espacios a la vez, y que un espacio aloje dos unidades.

Dominio de Personal: Pilotos, Guardias y Cobertura (RF-07, RF-08)

Figura 4

ER del dominio de personal

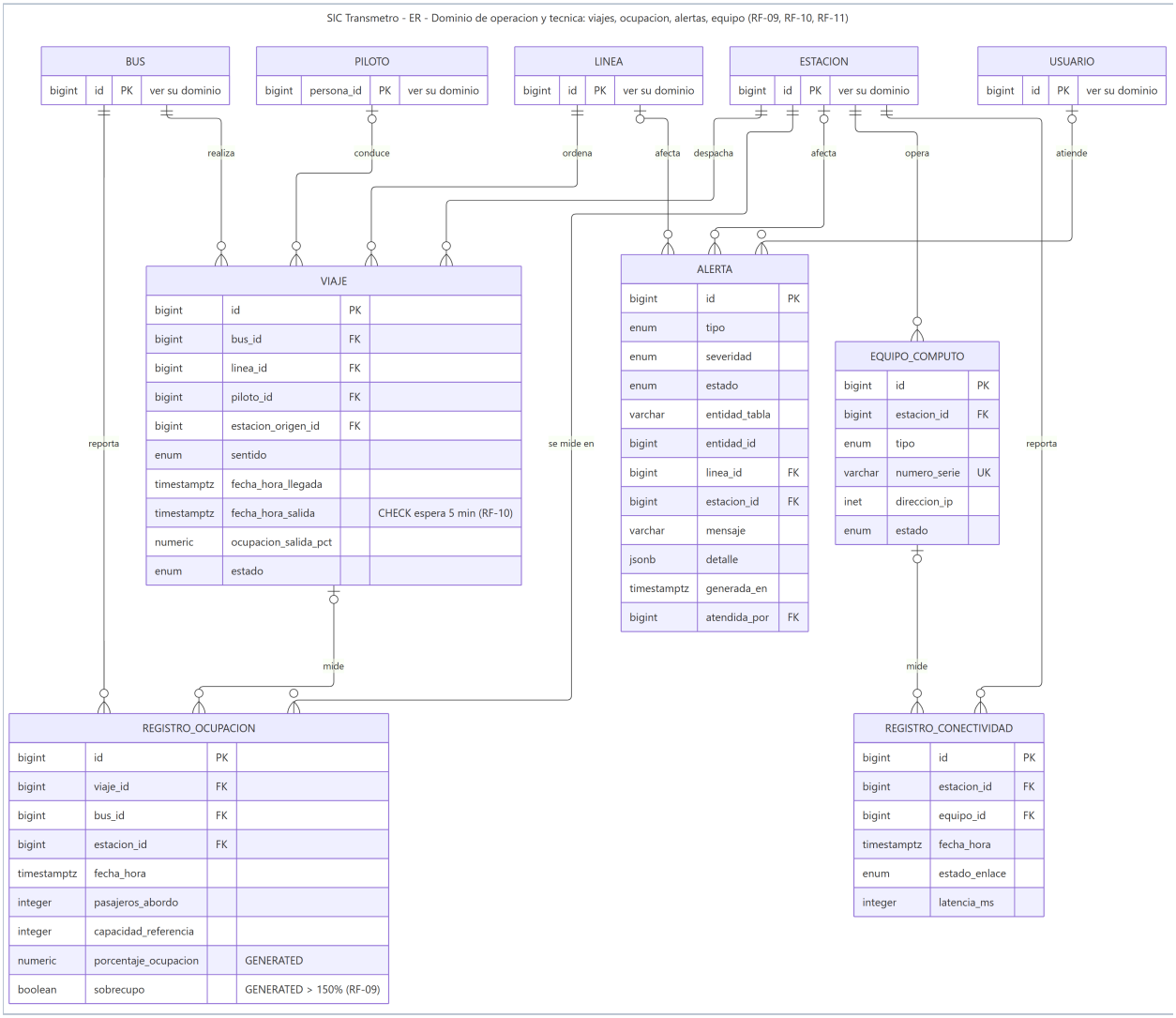


Nota. PERSONA es la generalización: PILOTO y GUARDIA usan como llave primaria el mismo persona_id, en una relación de uno a cero o uno. GUARDIA_ASIGNACION registra la cobertura por acceso, fecha y turno. Este dominio es el que da contenido a los módulos de personal, mi expediente y mi cobertura.

Dominio de Operación y Técnica (RF-09, RF-10, RF-11)

Figura 5

ER del dominio de operación

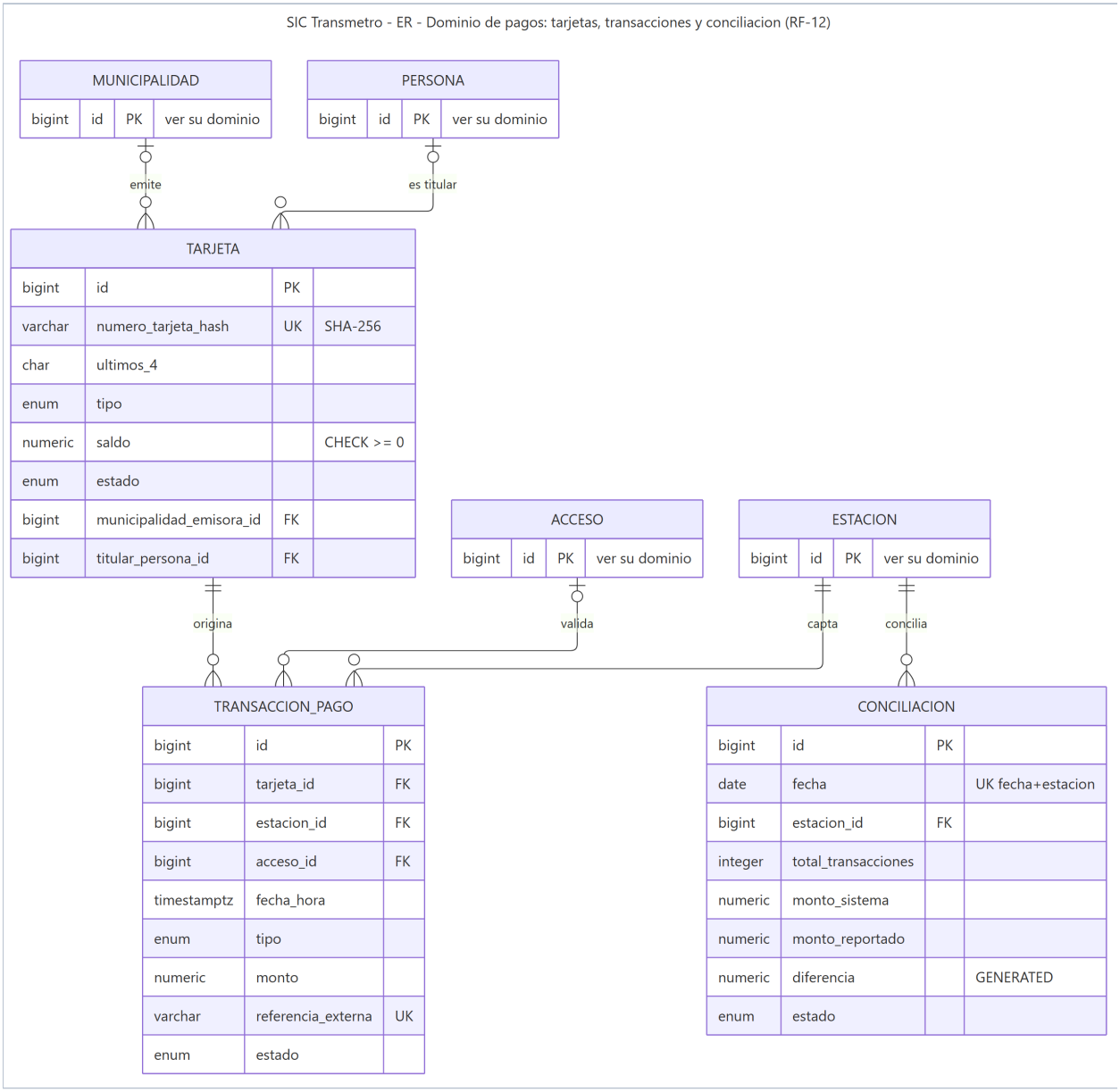


Nota. REGISTRO_OCUPACION calcula porcentaje_ocupacion y sobrecupo como columnas generadas, de modo que el valor no depende de quién escriba la fila. ALERTA es la bandeja única donde los disparadores dejan cada excepción operativa. Este dominio es el que da contenido al módulo de monitoreo y a la gráfica de ocupación del tablero.

Dominio de Pagos (RF-12)

Figura 6

ER del dominio de pagos

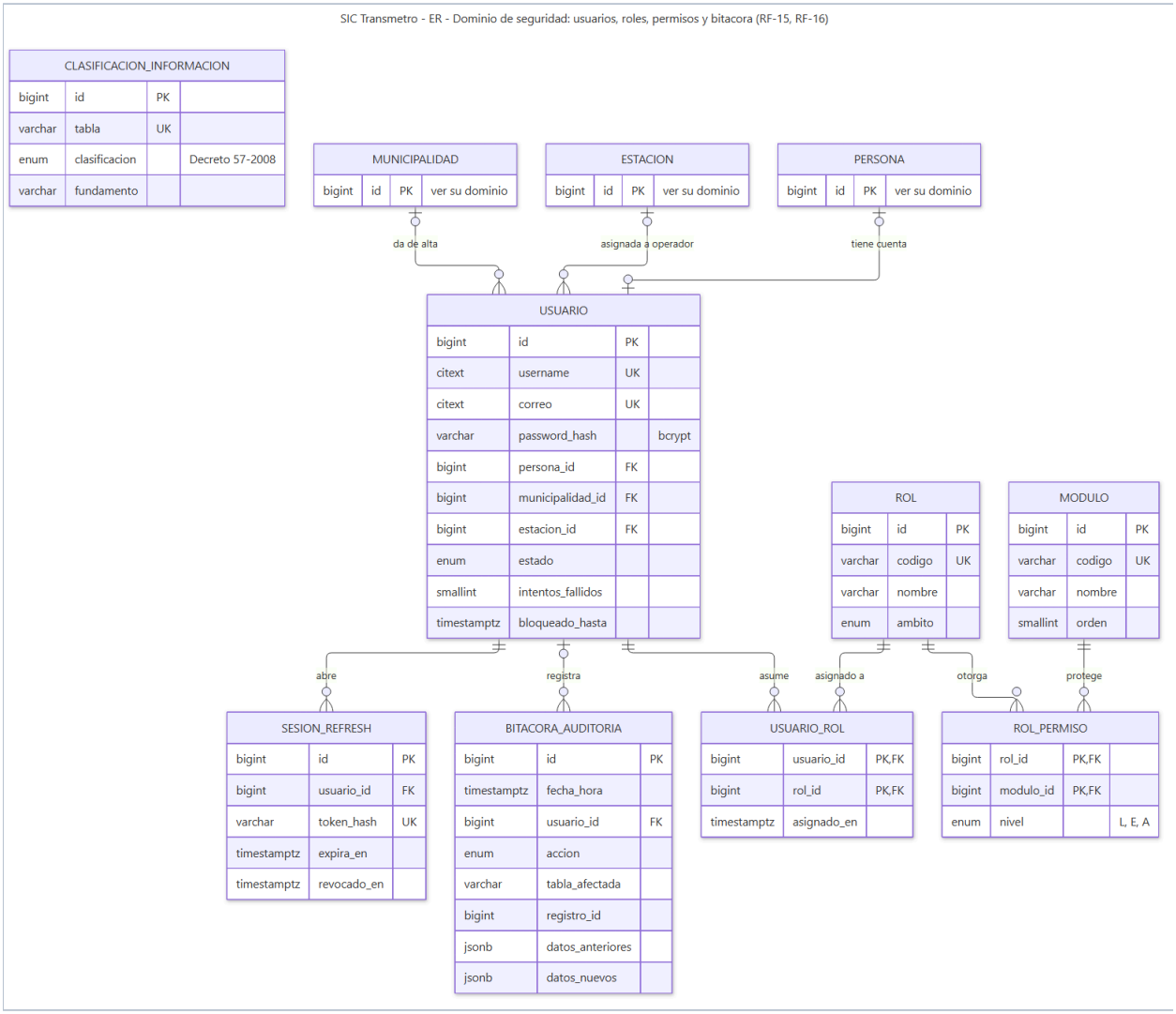


Nota. El número de tarjeta nunca se guarda en claro: solo su hash y los últimos cuatro dígitos. La referencia externa es única para que una transacción reenviada no se duplique.

Dominio de Seguridad (RF-15, RF-16)

Figura 7

ER del dominio de seguridad



Nota. ROL_PERMISO es la matriz de permisos normalizada, con el rol, el módulo y el nivel de lectura, escritura o administración. Si no existe la fila, el rol no tiene acceso a ese módulo.

Cardinalidades Principales

Tabla 18

Cardinalidades principales del modelo entidad relación

Relación	Cardinalidad	Regla
Municipalidad – Estación	1 : N	Toda estación pertenece a una municipalidad (RF-13)
Línea – Estación	M : N	Mediante LINEA_ESTACION, con orden y distancia (RF-01, RF-02)
Estación – Acceso	1 : N	Inventario de accesos (RF-03)
Bus – Línea	N : 1 vigente	Una sola asignación sin fecha de fin (RF-04)
Bus – Espacio de parqueo	1 : 1 vigente	Exclusión por periodo en ambos sentidos (RF-06)
Acceso – Guardia	M : N por fecha y turno	Mediante GUARDIA_ASIGNACION (RF-08)
Persona – Piloto o Guardia	1 : 0..1	Generalización
Viaje – Registro de ocupación	1 : N	Una medición por estación (RF-09)
Tarjeta – Transacción	1 : N	Registro de pagos (RF-12)
Usuario – Rol	M : N	Mediante USUARIO_ROL (RF-15)
Rol – Módulo	M : N con nivel	Mediante ROL_PERMISO (RF-15)
Usuario – Parámetro del sistema	1 : N	Deja constancia de quién modificó cada umbral

Nota. Elaboración propia.

Script de Base de Datos

El script completo tiene 2,521 líneas y crea 33 tablas, 12 vistas, 15 funciones, 29 disparadores y los datos de ejemplo. Se puede ejecutar varias veces: su primera sección elimina el esquema transmetro.

El archivo completo se entrega como entregables\02_Script_BD_completo.sql, junto con este documento. Se ejecuta así:

```
psql "postgresql://usuario:clave@host:5432/base" -v ON_ERROR_STOP=1 -f
02_Script_BD_completo.sql
```

También se puede abrir y ejecutar completo en pgAdmin o DBeaver. La contraseña de los usuarios de ejemplo es Transmetro2026*.

Estructura del Script

Tabla 19

Estructura del script de base de datos por sección

Sección	**Contenido**
00_drop_all	Elimina el esquema para poder ejecutar de nuevo
01_extensiones_dominios	Esquema, extensiones btree_gist, citext y pgcrypto, y 31 tipos enumerados
02_tablas_seguridad	Municipalidad, módulo, rol, matriz de permisos, usuario, sesión, bitácora y clasificación de información
03_tablas_red	Línea con su tarifa y horario, estación, recorrido, acceso y parámetros del sistema
04_tablas_flota	Bus, asignación a línea, predio, espacio y asignación de parqueo
05_tablas_personal	Persona, piloto, historial educativo, guardia, turno y cobertura
06_tablas_operacion	Alerta, viaje, registro de ocupación, equipo de cómputo y conectividad
07_tablas_pagos	Tarjeta, transacción de pago y conciliación
08_reglas_negocio	Funciones y disparadores de reglas de negocio y auditoría
09_indices	37 índices para sostener el tiempo de respuesta menor a 3 segundos
10_vistas	12 vistas de consulta y reportes
11_seed	Catálogos obligatorios, corredores reales y datos de ejemplo con historial de operación

Nota. La sección 11_seed creció respecto de la edición anterior: además de los catálogos, carga los siete corredores con su tarifa y horario, 26 estaciones con sus coordenadas, 170 unidades, 15 pilotos, 14 guardias, la cobertura semanal de los accesos y 23 horas de viajes con sus mediciones de ocupación, de modo que el tablero y las gráficas tengan contenido desde el primer arranque.

Reglas de Negocio dentro de la Base

Las reglas viven en PostgreSQL, no en la interfaz REST. Así ningún cliente puede saltárselas: ni la aplicación web, ni un guion de carga, ni una consulta manual.

Tabla 20

Reglas de negocio implementadas en el motor de base de datos

Requerimiento	**Mecanismo**
RF-04 Una unidad en una sola línea activa	Restricción de exclusión sobre la unidad y el periodo, más índice único parcial
RF-05 Rango mínimo y máximo de unidades	Disparador que abre o cierra la alerta de rango de unidades
RF-06 Ninguna unidad sin parqueo	Dos restricciones de exclusión y un disparador de alerta
RF-08 Cobertura mínima de guardias	Disparador que compara contra el mínimo declarado en el acceso
RF-09 Sobrecupo del 50 % adicional	Columnas generadas y disparador de alerta
RF-10 Espera de 5 minutos	Restricción de verificación sobre la fila del viaje
RF-16 Bitácora de auditoría	Disparador genérico de auditoría sobre 14 tablas

Nota. Las siete reglas anteriores son las llamadas reglas duras: su violación produce datos inconsistentes, no solo una molestia para el usuario.

RF-04, una unidad solo puede estar activa en una línea a la vez:

```
CREATE TABLE bus_linea_asignacion (
    id           bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    bus_id       bigint      NOT NULL REFERENCES bus(id)   ON DELETE
CASCADE,
    linea_id     bigint      NOT NULL REFERENCES linea(id) ON DELETE
RESTRICT,
    fecha_inicio timestamptz NOT NULL DEFAULT now(),
    fecha_fin    timestamptz,
    motivo       varchar(250),
    CONSTRAINT ck_bus_asig_fechas CHECK (fecha_fin IS NULL OR fecha_fin
> fecha_inicio),
    CONSTRAINT ex_bus_una_linea_a_la_vez EXCLUDE USING gist (
        bus_id                                  WITH =,
        tstzrange(fecha_inicio, fecha_fin, '[)') WITH &&
    )
);

CREATE UNIQUE INDEX ux_bus_asignacion_vigente
    ON bus_linea_asignacion (bus_id)
    WHERE fecha_fin IS NULL;
```

RF-09, ocupación y sobrecupo calculados por la base:

```
porcentaje_ocupacion numeric(6,2) GENERATED ALWAYS AS (
    round(pasajeros_abordo * 100.0 / capacidad_referencia, 2)
) STORED,
sobrecupo boolean GENERATED ALWAYS AS (
    pasajeros_abordo > capacidad_referencia * 1.5
) STORED,
```

RF-10, espera mínima de 5 minutos con menos del 25 % de ocupación:

```
CONSTRAINT ck_viaje_espera_minima CHECK (
    fecha_hora_salida IS NULL
    OR ocupacion_salida_pct IS NULL
    OR ocupacion_salida_pct >= 25
    OR (fecha_hora_salida - fecha_hora_llegada) >= INTERVAL '5 minutes'
)
```

Esta restricción es la que sostiene el módulo de monitoreo: la pantalla muestra la cuenta regresiva y deshabilita el botón de despacho, pero quien garantiza la regla es la base de datos. Si alguien llamara a la interfaz REST directamente, el motor rechazaría la operación igual.

RF-05, alerta automática por rango de unidades, extracto:

```
IF v_activas < v_linea.unidades_minimas THEN
    PERFORM fn_alerta_abrir(
        'rango_unidades', 'critica', 'linea', p_linea_id, p_linea_id,
NULL,
        format('La linea %s opera con %s unidades, por debajo del minimo
de %s.',
               v_linea.codigo, v_activas, v_linea.unidades_minimas));
ELSIF v_activas > v_linea.unidades_maximas THEN
    PERFORM fn_alerta_abrir('rango_unidades', 'advertencia', ...);
ELSE
    PERFORM fn_alerta_cerrar('rango_unidades', 'linea', p_linea_id);
END IF;
```

Resultado de la Ejecución de Prueba

El script se ejecutó dos veces seguidas en PostgreSQL 18.3 mediante PGlite 0.5.8, la más reciente el 19 de octubre de 2026, sin errores. Después se probaron las reglas con datos válidos e inválidos.

Tabla 21

Resultado de las pruebas de reglas de negocio

Regla	Prueba	Resultado
RF-04	Asignar a la L2 una unidad que ya está en la L1	Rechazado por ex_bus_una_linea_a_la_vez
RF-06	Asignar una segunda unidad a un espacio ocupado	Rechazado por ex_espacio_un_bus
RF-10	Despachar con 20 % de ocupación a los 3 minutos	Rechazado por ck_viaje_espera_minima
RF-10	Despachar con 20 % de ocupación a los 6 minutos	Aceptado
RF-09	256 pasajeros en una unidad de capacidad 160	Alerta crítica de sobrecupo, 160 %
RF-05	Línea operando por debajo de su mínimo de unidades	Alerta crítica de rango
RF-08	Acceso con cobertura de guardias incompleta	Alerta de cobertura con el detalle de turnos
RF-16	Modificar el nombre de una línea	Registro en bitácora con usuario, acción y campos cambiados
RF-02	Longitud de la red en ambos sentidos	52.5 km, coincidentes con la suma de los tramos

Nota. Las pruebas de reglas y sus resultados detallados figuran como casos CP-01 a CP-38 en el plan de pruebas del proyecto.

Diagramas UML

Son 15 diagramas de 7 tipos, que cubren la vista funcional, la estática, la dinámica y la física del sistema.

Tabla 22

Diagramas UML elaborados por tipo

Tipo	Diagramas	Qué muestra
Casos de uso	6, uno general y uno por actor	Qué hace cada rol en el sistema
Clases	1	Modelo de dominio: clases, atributos, métodos y asociaciones
Secuencia	3	Interacción entre interfaz, servidor y base en tres flujos críticos
Actividades	1	Flujo de llegada, ocupación y despacho de una unidad
Estados	2	Ciclo de vida de la unidad y de la alerta
Componentes	1	Módulos de la aplicación web, del servidor y de la base de datos
Despliegue	1	Nodos físicos y protocolos en la nube

Nota. Los diagramas se generan con Mermaid CLI a partir de fuentes de texto que se versionan junto con el código.

Diagramas de Casos de Uso

Hay 5 actores, uno por rol de la matriz de permisos, y 24 casos de uso. El administrador general hereda todos los casos del administrador municipal. En color distinto se marcan los casos incluidos o extendidos, que el sistema ejecuta de forma automática.

Figura 8

Casos de uso general

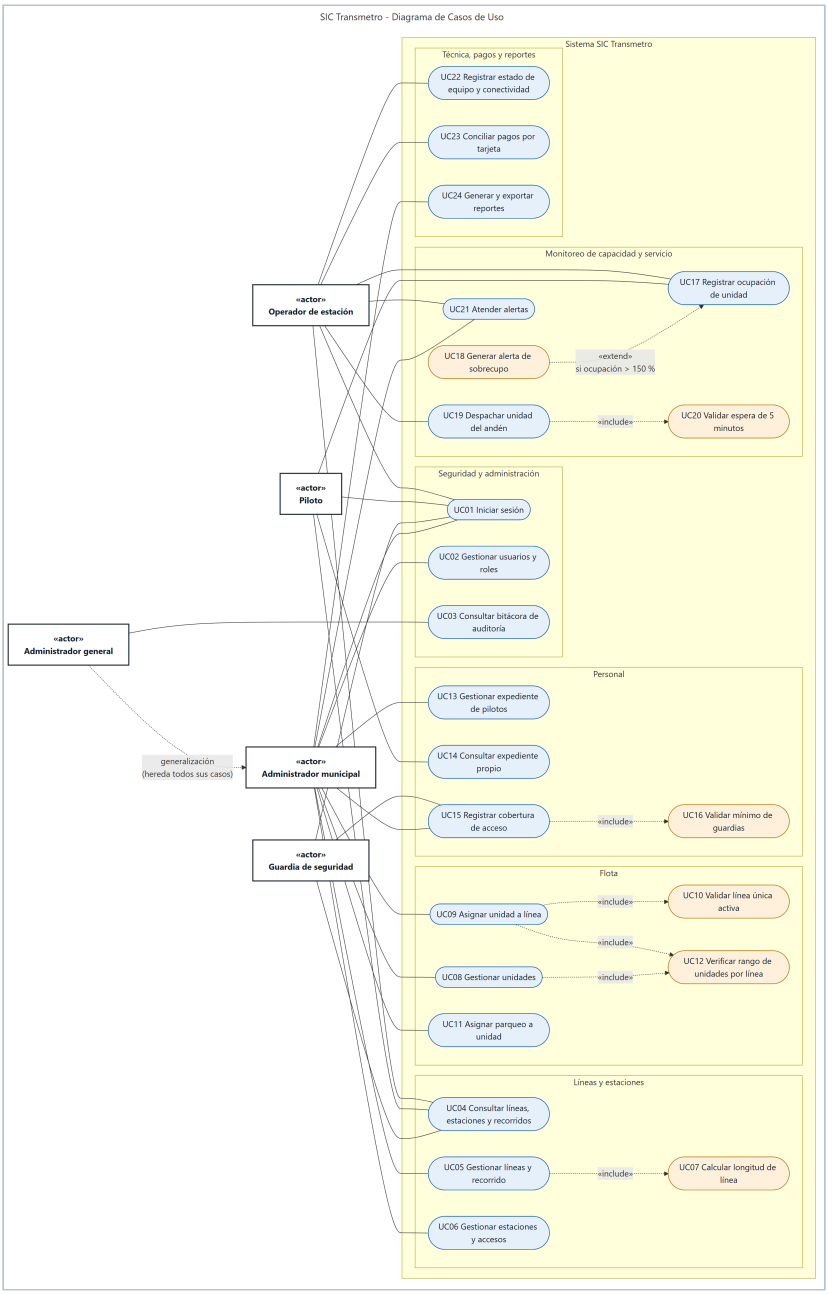
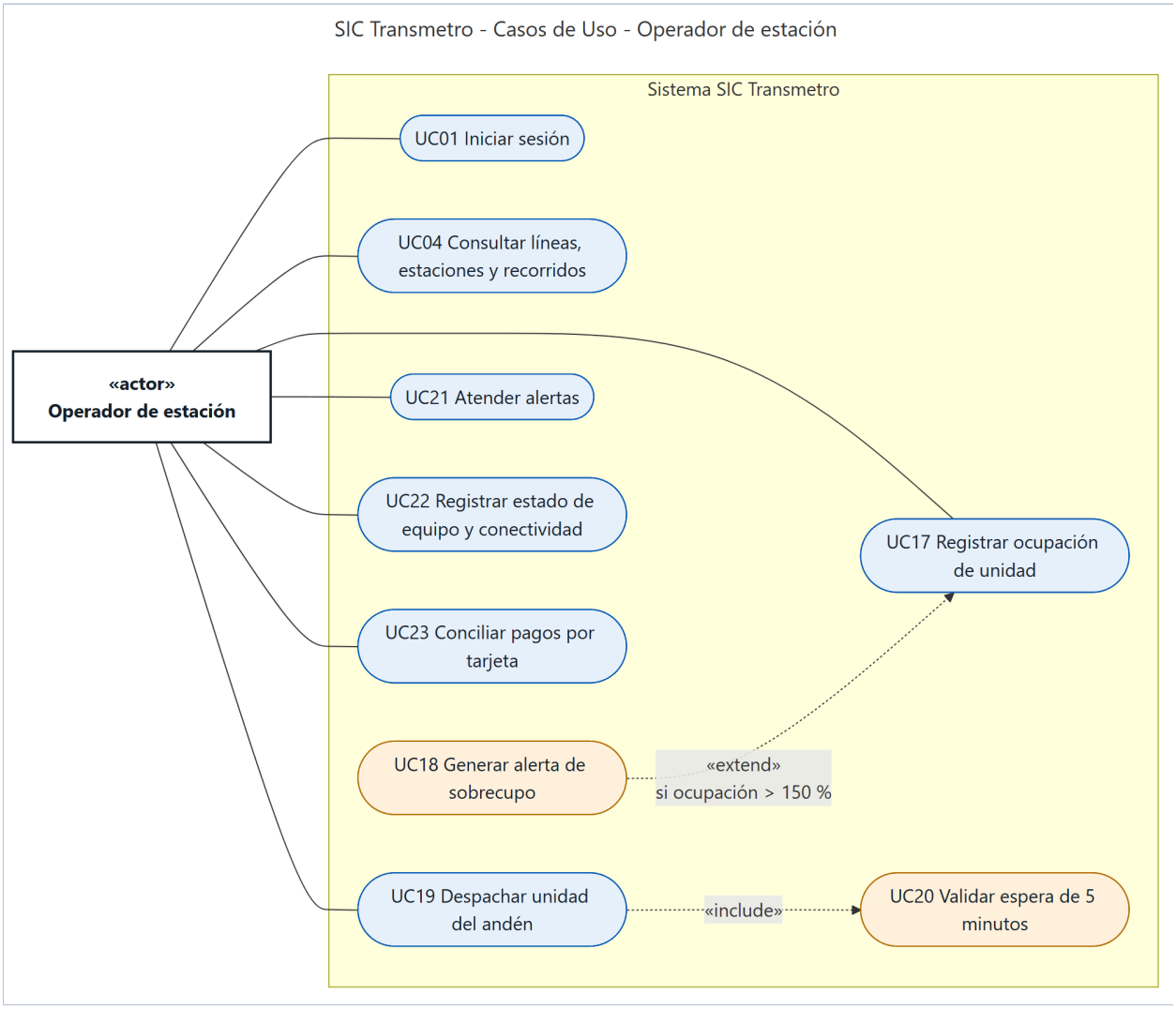


Figura 9

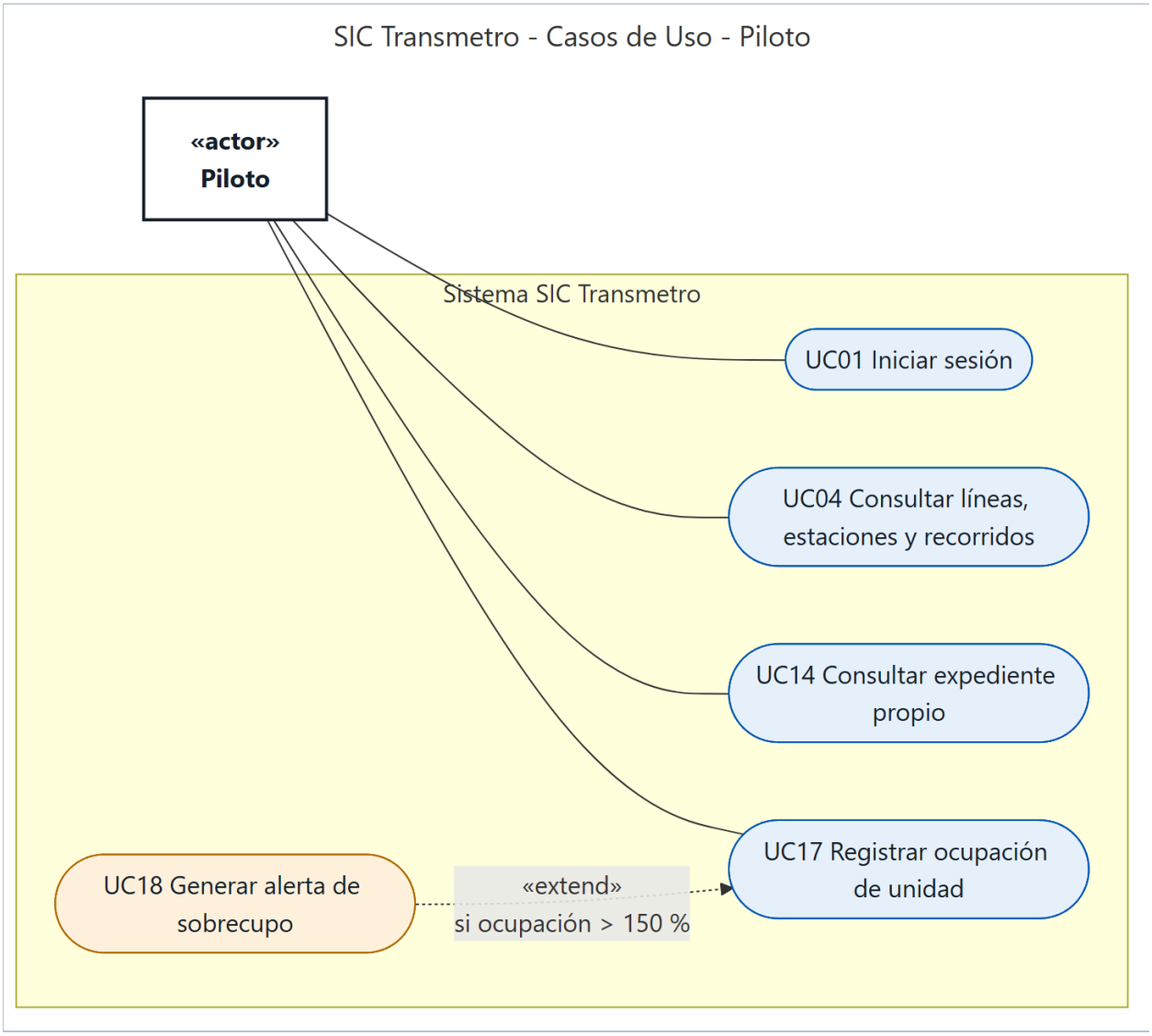
Casos de uso del operador de estación



Nota. Los casos de este actor corresponden al módulo de monitoreo, hoy construido: registrar la llegada de la unidad, medir su ocupación y despacharla.

Figura 10

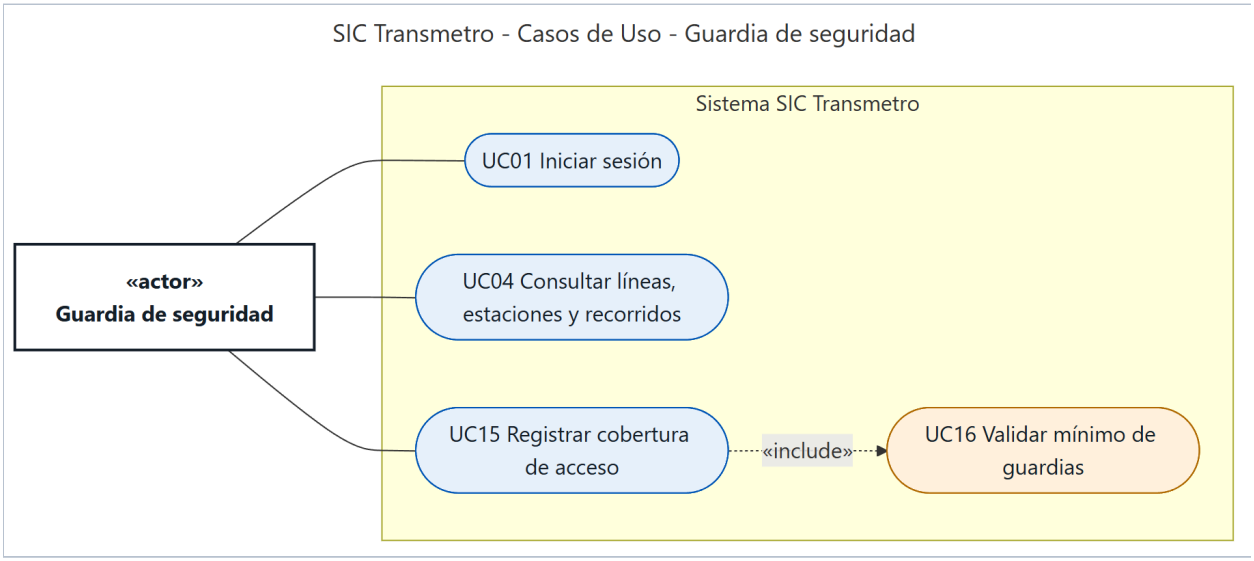
Casos de uso del piloto



Nota. Los casos de consulta de expediente y de viajes propios corresponden al módulo mi expediente, hoy construido.

Figura 11

Casos de uso del guardia de seguridad



Nota. Los casos de consulta de turnos y registro de asistencia corresponden al módulo mi cobertura, hoy construido.

Figura 12

Casos de uso del administrador municipal

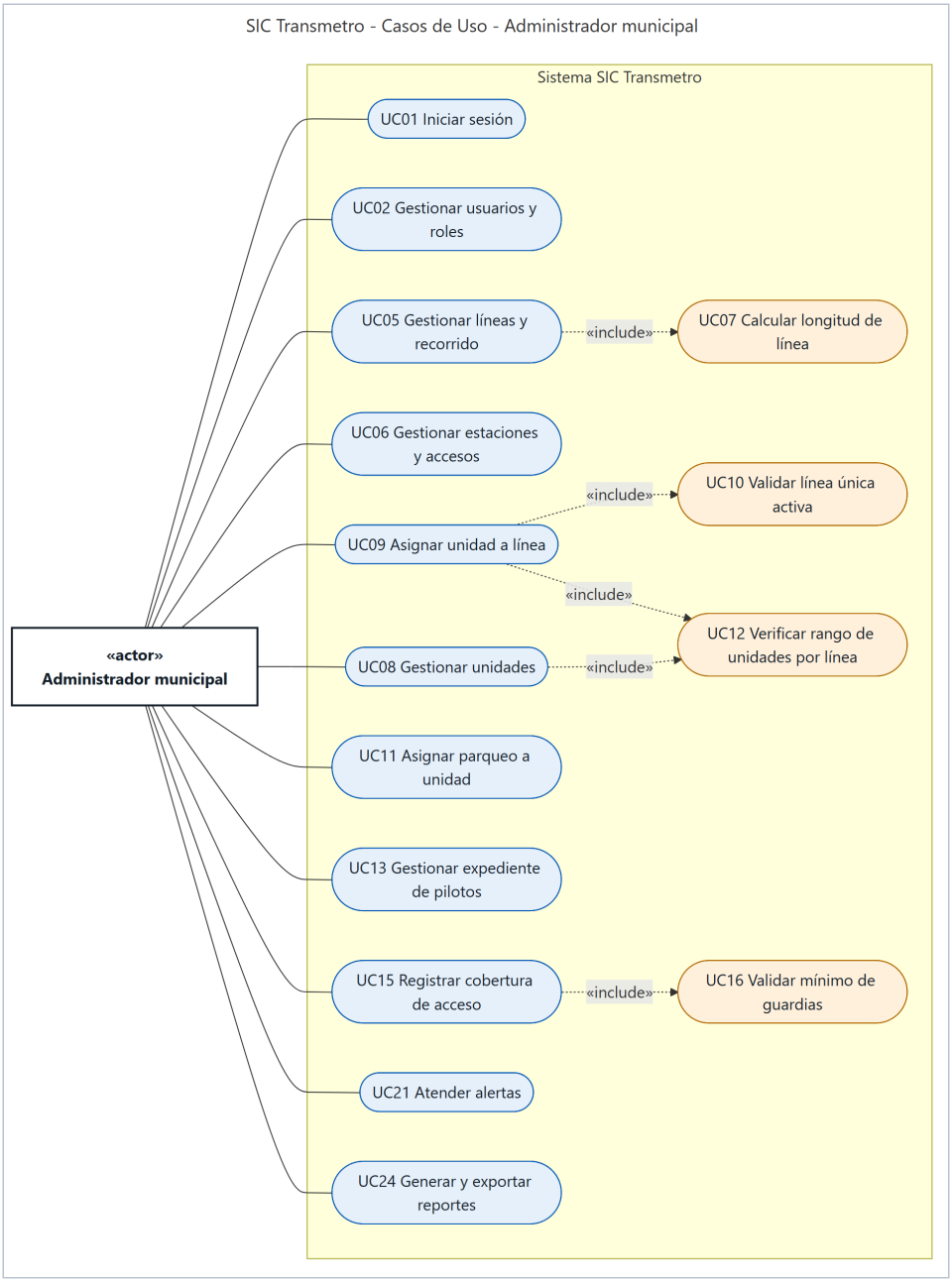


Figura 13

Casos de uso del administrador general

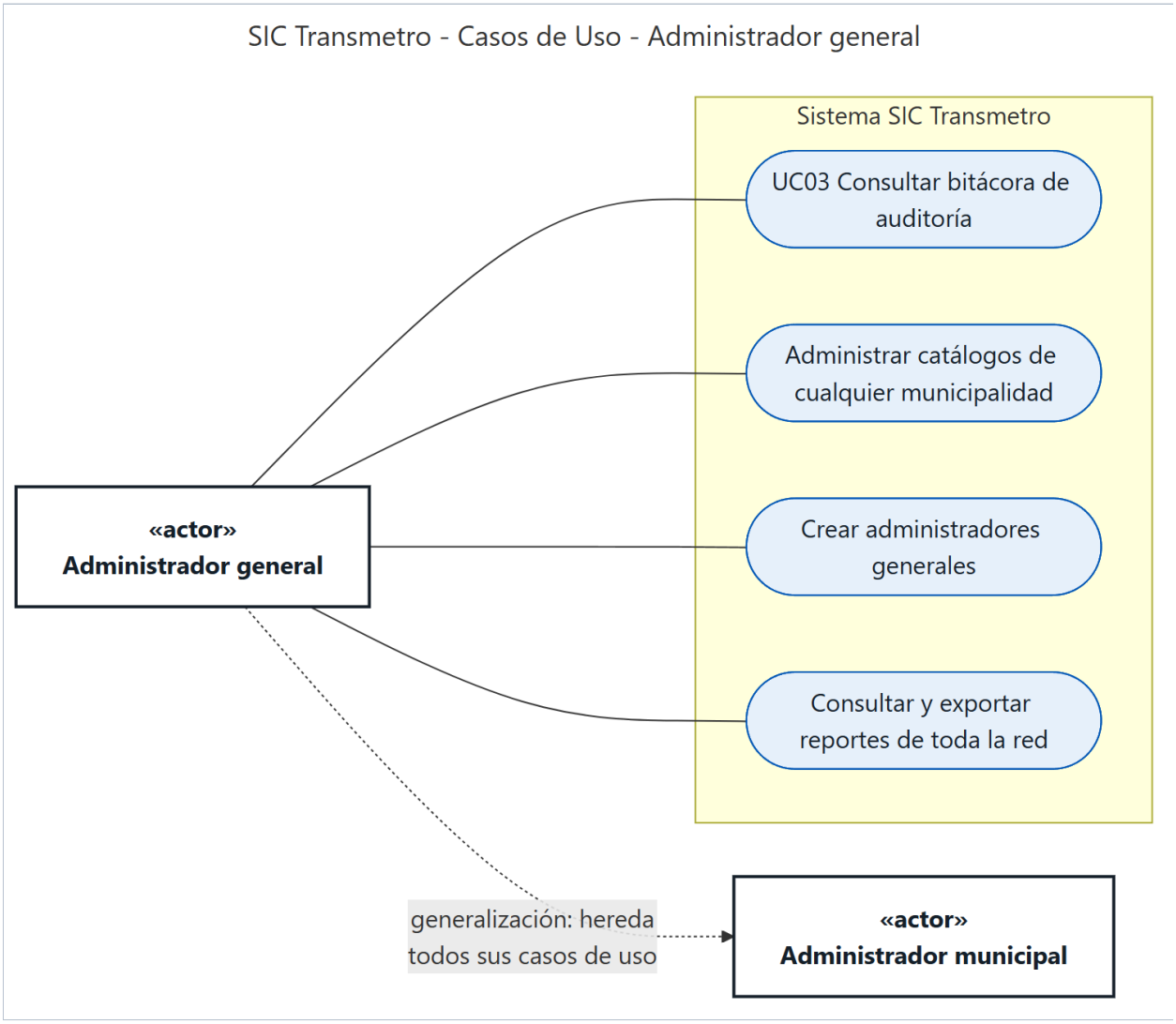
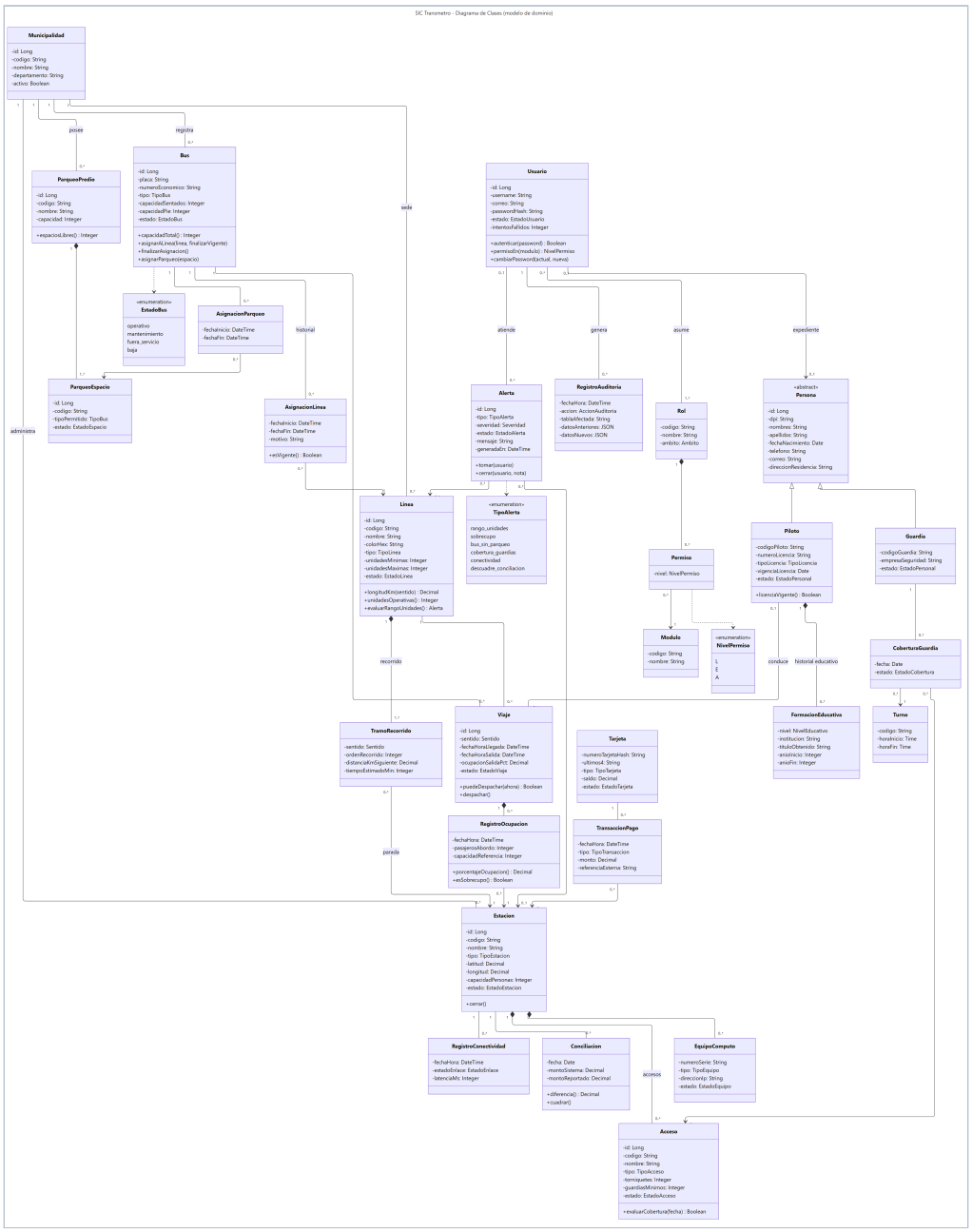


Diagrama de Clases

Figura 14

Diagrama de clases del modelo de dominio



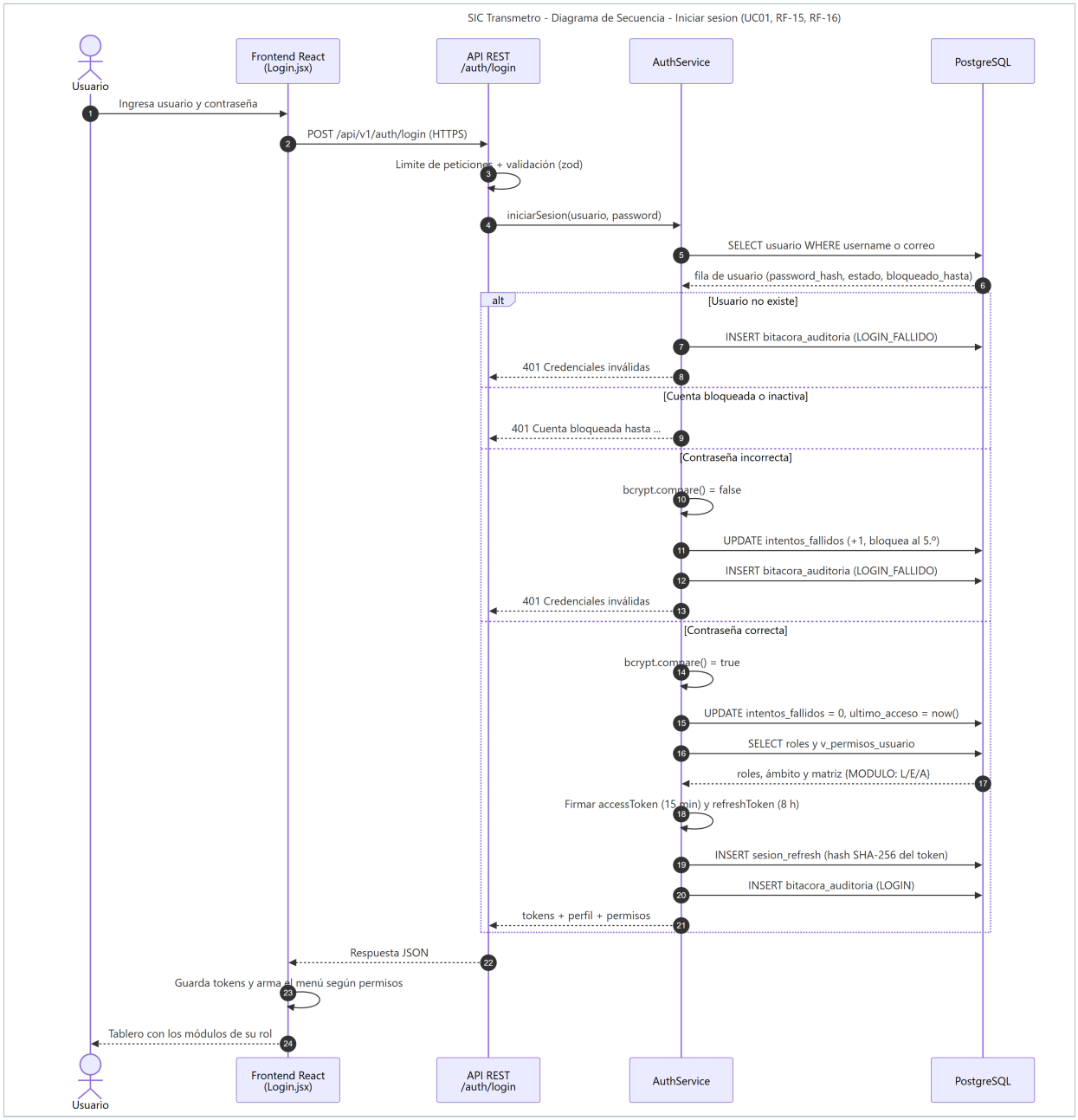
Nota. Persona es abstracta y se especializa en Piloto y Guardia. Las composiciones, con rombo relleno, marcan las partes que no existen sin su todo: el recorrido de una línea, los accesos

de una estación y los espacios de un predio. Los métodos reflejan las reglas: asignar una unidad a una línea, determinar si un viaje puede despacharse y determinar si una medición es sobrecupo.

Diagramas de Secuencia

Figura 15

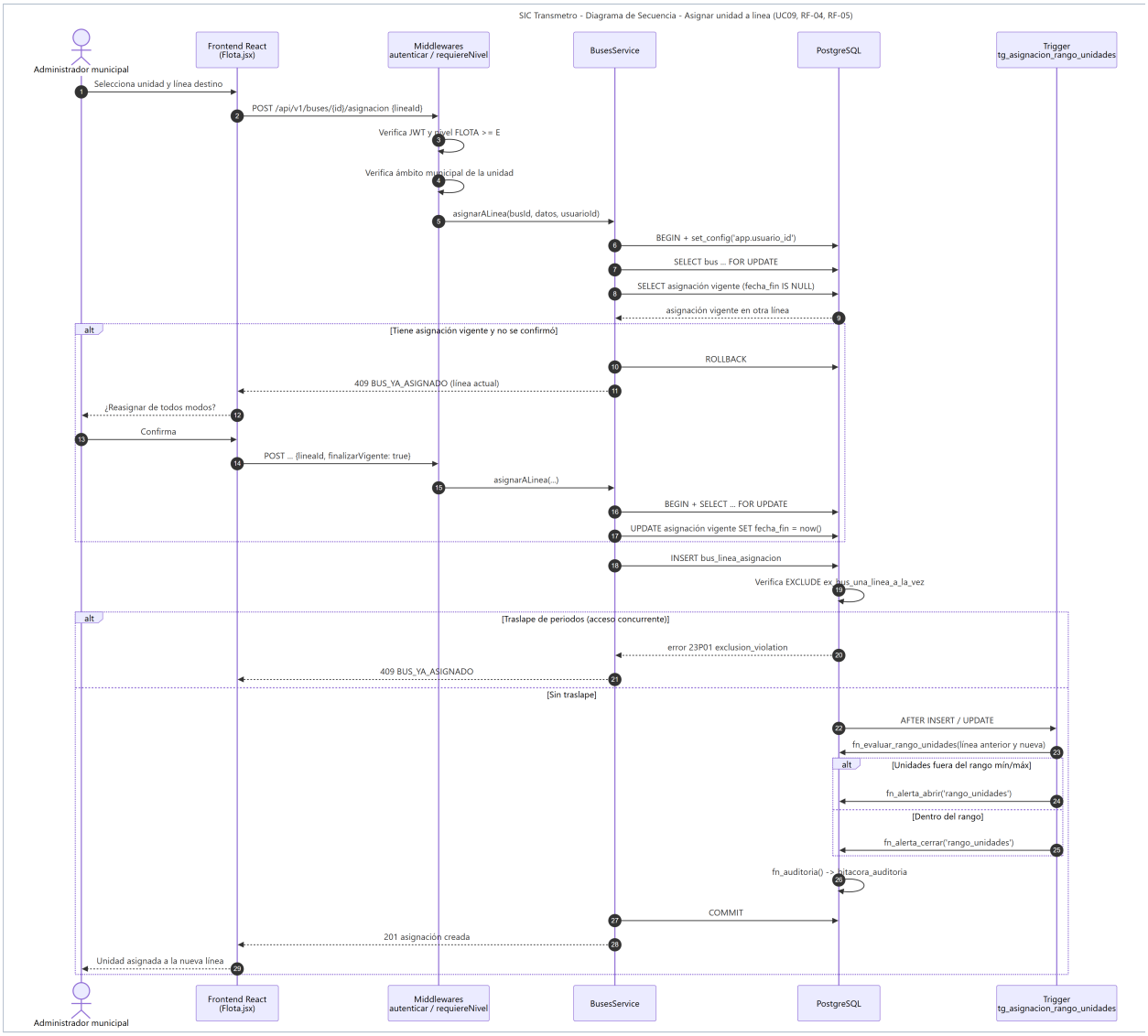
Secuencia de inicio de sesión (RF-15, RF-16)



Nota. El sistema bloquea la cuenta 15 minutos después de cinco intentos fallidos. Cada intento, exitoso o no, queda en la bitácora. El token de acceso lleva los permisos del usuario, con los que la interfaz arma el menú.

Figura 16

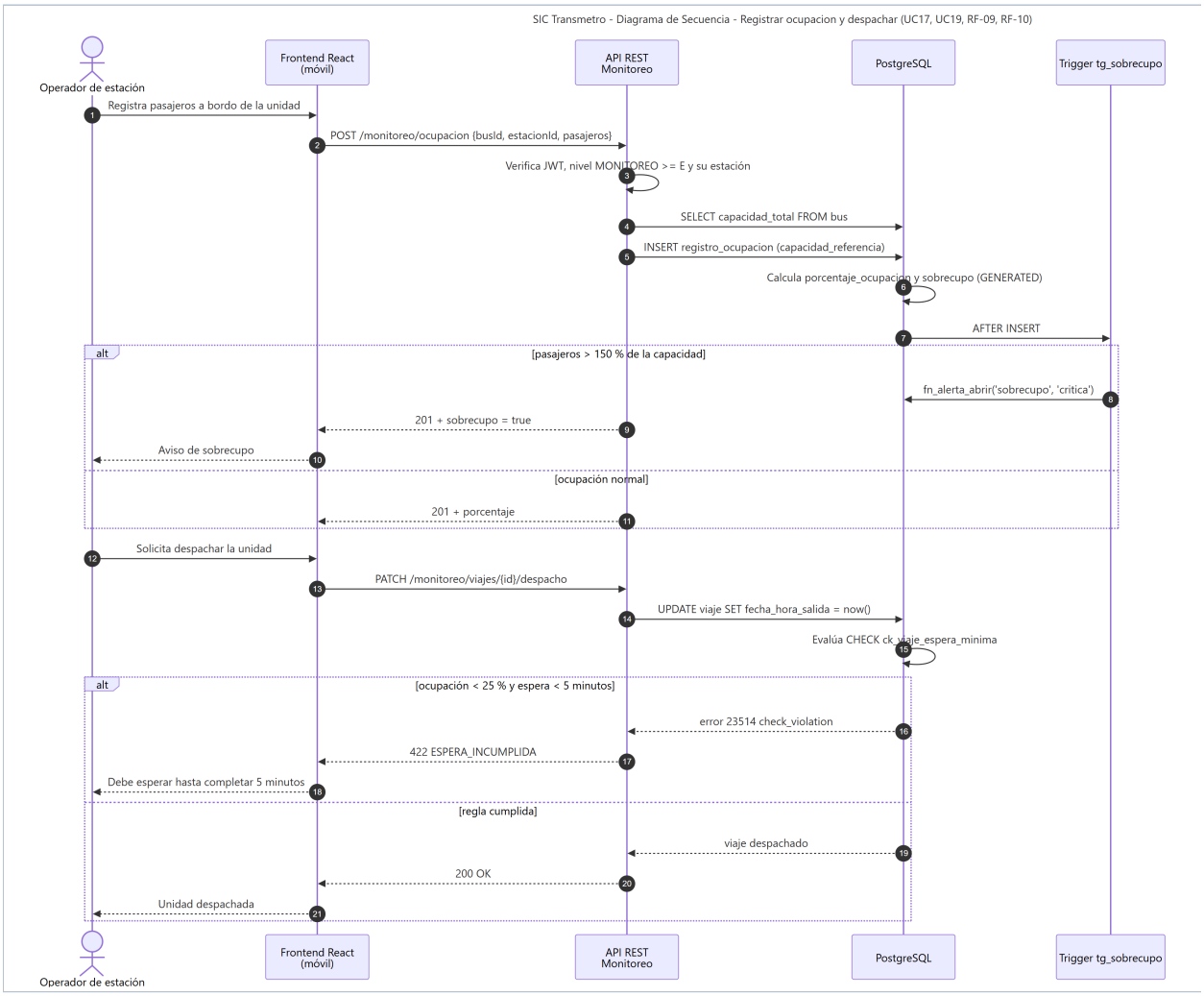
Secuencia de asignación de unidad a línea (RF-04, RF-05)



Nota. La regla se valida en tres capas: la interfaz pide confirmar la reasignación, el servidor bloquea la fila y la base rechaza cualquier traslape con la restricción de exclusión. Si dos personas asignan la misma unidad al mismo tiempo, una de las dos operaciones falla.

Figura 17

Secuencia de ocupación y despacho (RF-09, RF-10)

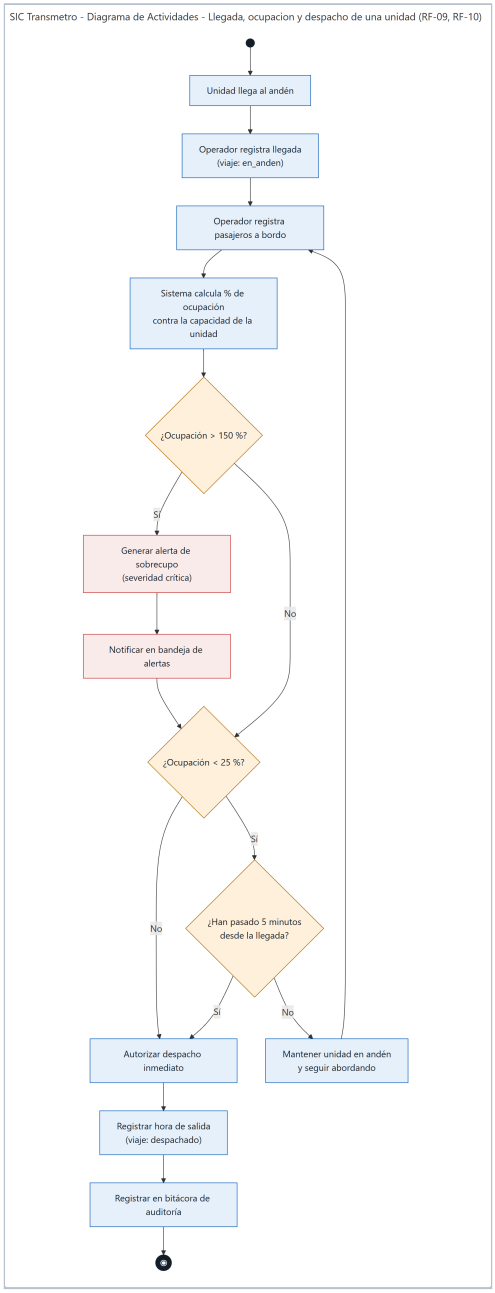


Nota. Este flujo corresponde al módulo de monitoreo, que en la edición anterior de este documento figuraba como planificado y hoy está construido y verificado con los casos CP-27 a CP-31 del plan de pruebas.

Diagrama de Actividades

Figura 18

Actividades del despacho de una unidad (RF-09, RF-10)



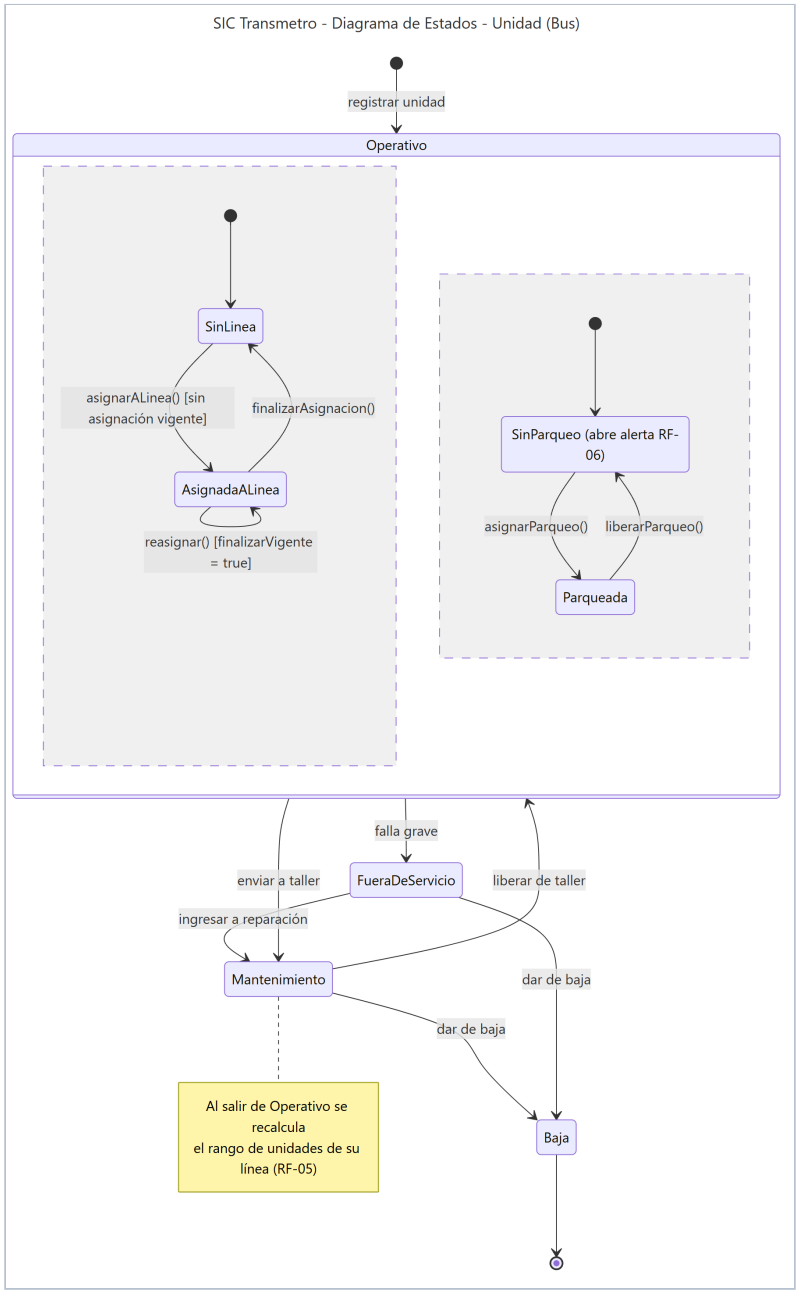
Nota. Si la ocupación pasa del 150 % se genera una alerta, pero la unidad sí puede salir. Si es menor al 25 %, la unidad espera en el andén hasta completar cinco minutos desde su llegada.

La pantalla de monitoreo presenta esa espera como una cuenta regresiva.

Diagramas de Estados

Figura 19

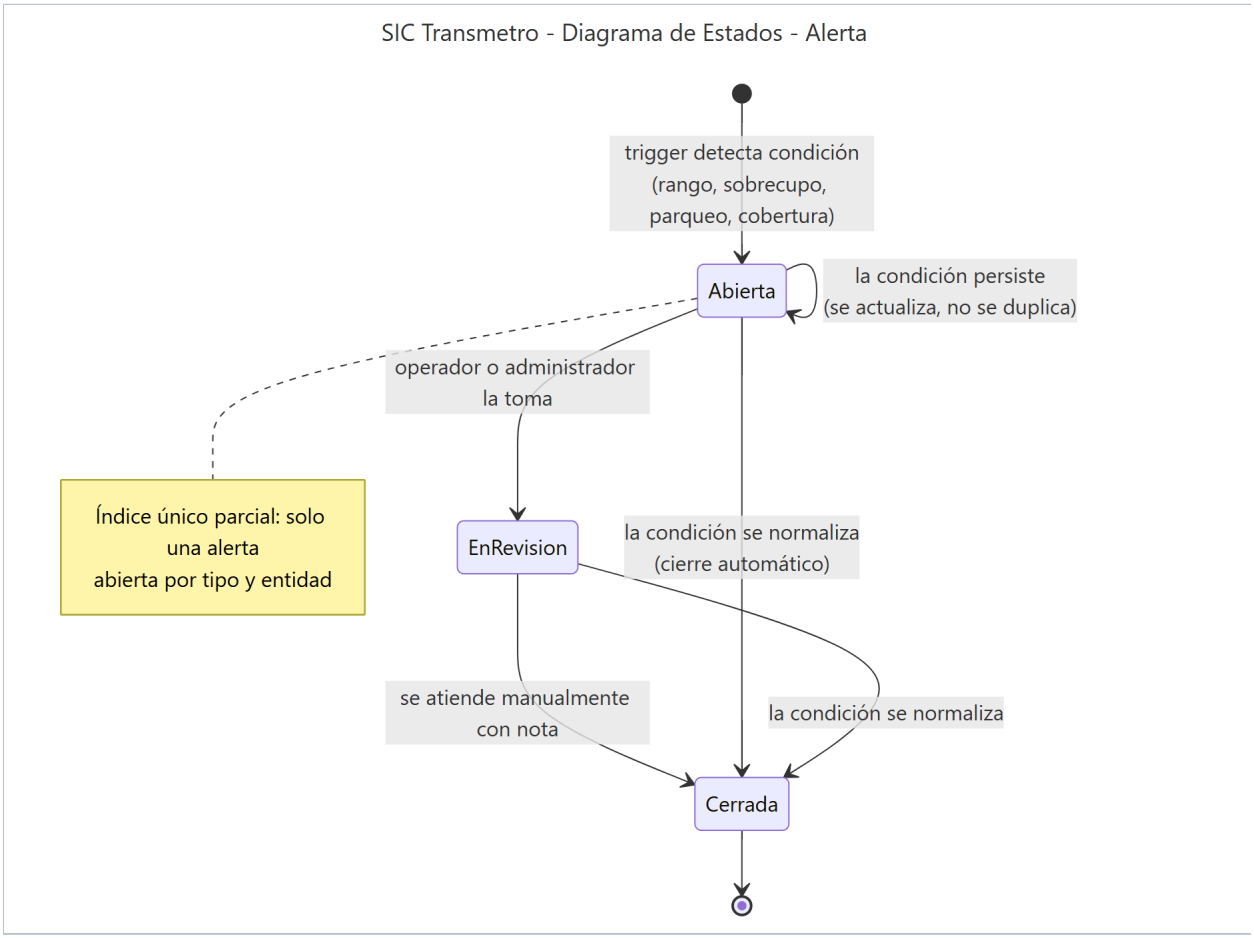
Estados de la unidad



Nota. Mientras está operativa, la unidad tiene dos regiones concurrentes: su asignación a línea y su parqueo. Si sale del estado operativo, se recalcula el rango de unidades de su línea (RF-05).

Figura 20

Estados de la alerta

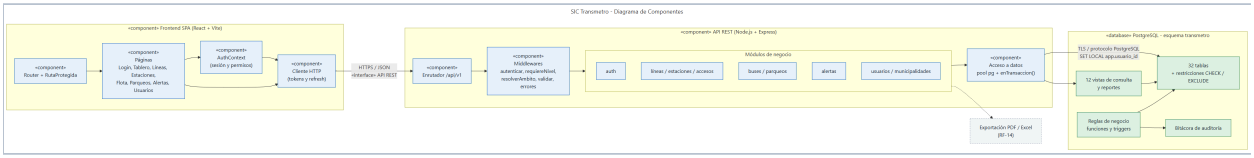


Nota. Una alerta se cierra de dos formas: manualmente, con una nota, o automáticamente cuando la condición se normaliza. Solo puede haber una alerta abierta por tipo y por entidad.

Diagrama de Componentes

Figura 21

Diagrama de componentes

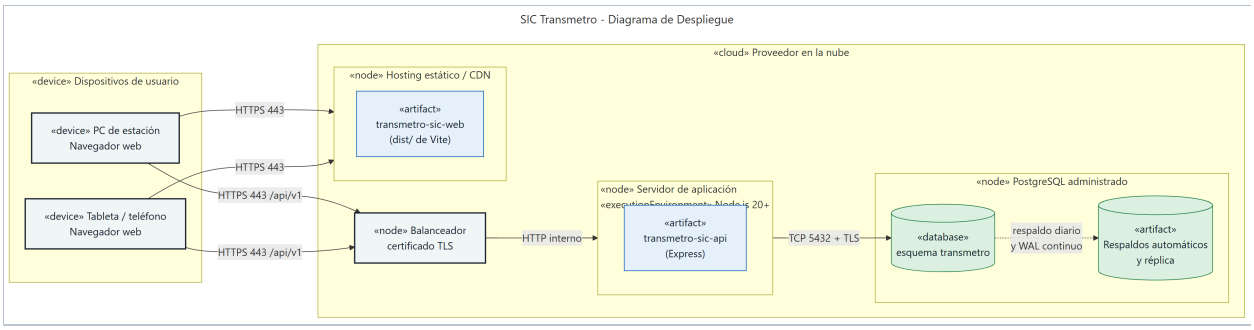


Nota. Las tres capas son independientes: la aplicación web solo conoce la interfaz REST y el servidor solo conoce la base. Esto cumple el requisito no funcional de capas desacopladas. Los módulos de monitoreo, personal y tablero se incorporaron al servidor sin modificar esta estructura.

Diagrama de Despliegue

Figura 22

Diagrama de despliegue



Nota. Toda la comunicación externa va por HTTPS en el puerto 443, y la conexión a la base usa TLS. El servicio administrado de PostgreSQL hace respaldos diarios y guarda el registro continuo de transacciones, lo que cubre el requisito de respaldo y recuperación ante desastres.

Parte IV. Documentación Funcional del Sistema

El Sistema de Información y Control del Transmetro (SIC Transmetro) se encuentra en ejecución. Este documento describe qué hace cada módulo de la aplicación y cómo ponerla en marcha. Reemplaza a la versión del 9 de octubre de 2026 e incorpora los módulos construidos después de esa fecha: monitoreo de capacidad y servicio, personal, mi expediente, mi cobertura y el tablero con mapa de la red e indicadores gráficos.

Los datos de catálogo reproducen los corredores reales del Transmetro de la ciudad de Guatemala y municipios vecinos. Cada línea registra su tarifa, su horario de operación y sus frecuencias de hora pico y hora valle, de modo que el sistema ya no trabaja con líneas genéricas sino con el servicio tal como opera.

Tabla 23

Tamaño de la red cargada en el sistema

Elemento	**Cantidad**
Líneas activas	7
Estaciones operativas	26
Longitud de la red	52.5 km
Accesos registrados	39
Unidades de la flota	170 (164 operativas y 6 en taller)
Espacios de parqueo	300 en 3 predios
Pilotos y guardias	15 pilotos y 14 guardias
Historial de operación cargado	69 viajes y 69 mediciones de ocupación en 23 horas

Nota. Cifras consultadas directamente en la base de datos el 19 de octubre de 2026. El historial de operación se carga con los datos de ejemplo para que el tablero y las gráficas tengan contenido desde el primer arranque.

Puesta en Marcha

El sistema consta de tres procesos que se ejecutan en paralelo. La base de datos utiliza PGlite, una compilación de PostgreSQL que funciona sin instalar un servidor, por lo que el sistema arranca en cualquier equipo con Node.js.

```
cd 02_backend && npm run db:local     Base de datos en el puerto 5433
cd 02_backend && npm run dev          API REST en el puerto 4000
cd 03_frontend && npm run dev         Aplicacion web en el puerto 5174
```

Luego se ingresa a http://localhost:5174. El comando `npm run db:local:reset` restablece la base al estado inicial.

Tabla 24

Usuarios de prueba cargados en el sistema

Usuario	Rol	Ámbito de datos	Módulos propios
admin.general	Administrador general	Toda la red	Todos
admin.muni.gt	Administrador municipal	Municipalidad de Guatemala	Catálogos, flota, parqueos, personal y usuarios
operador.e103	Operador de estación	Estación El Frutal	Monitoreo y alertas
piloto.001	Piloto	Sus propios registros	Mi expediente
guardia.001	Guardia de seguridad	Sus propios registros	Mi cobertura

Nota. La contraseña de los cinco usuarios es Transmetro2026*. El menú se arma con la matriz de permisos y con el rol, de modo que cada perfil ve únicamente sus opciones.

Módulos Construidos

El sistema pasó de ocho a doce módulos con interfaz. La tabla siguiente describe cada uno y marca con la palabra *nuevo* los que se incorporaron en esta entrega.

Tabla 25

Módulos en funcionamiento y su propósito

Módulo	Qué hace	Requerimientos	Quién lo usa
Autenticación	Valida usuario y contraseña, emite el token de sesión, bloquea la cuenta tras cinco intentos fallidos y registra cada ingreso en la bitácora	RF-15, RF-16	Todos los roles
Tablero (renovado)	Mapa de la red, cinco indicadores y tres gráficas: unidades por línea frente a su rango, alertas abiertas por tipo y ocupación promedio por hora	RF-02, RF-05, RF-06, RF-09	Todos los roles, con el alcance de cada uno
Monitoreo (nuevo)	Andén de la estación: registra la llegada de cada unidad, su ocupación y su despacho, y muestra en vivo el tiempo que falta para cumplir la espera obligatoria	RF-09, RF-10	Operador de estación
Personal (nuevo)	Expedientes de pilotos con su licencia e historial educativo, listado de guardias y turnos que no alcanzan la cobertura mínima	RF-07, RF-08	Administrador municipal y general
Mi expediente (nuevo)	El piloto consulta sus datos, su licencia, su historial educativo y sus viajes; no accede al expediente de otros	RF-07	Piloto
Mi cobertura (nuevo)	El guardia consulta sus turnos por acceso y registra su asistencia; solo puede actuar sobre los suyos	RF-08	Guardia de seguridad
Líneas	Administra el catálogo de líneas con su tarifa, horario y frecuencias, y el recorrido de cada una: orden de estaciones, distancia entre ellas y longitud total calculada	RF-01, RF-02	Administrador municipal y general
Estaciones	Administra el catálogo de estaciones, su municipalidad responsable y el inventario de accesos con su mínimo de guardias	RF-01, RF-03, RF-13	Administrador municipal y general
Flota	Registra las unidades y administra su asignación a una línea. Impide que una unidad quede asignada a dos líneas y exige confirmación para reasignar	RF-04	Administrador municipal
Parqueos	Administra predios y espacios, asigna un espacio a cada unidad y lista las unidades operativas que quedaron sin parqueo	RF-06	Administrador municipal

Alertas	Bandeja de excepciones que la base de datos genera sola. Permite tomar una alerta y cerrarla con nota	RF-05, RF-06, RF-08, RF-09	Operador, administrador municipal y general
Usuarios	Administra cuentas, asigna roles y muestra la matriz de permisos vigente	RF-15	Administrador municipal y general

Nota. Los módulos de administración técnica, conciliación de pagos y reportería gerencial tienen sus tablas y reglas creadas en la base de datos, pero aún no cuentan con interfaz.

Módulos Nuevos en Detalle

Monitoreo de Capacidad y Servicio

El módulo de monitoreo es la pantalla de trabajo del operador de estación y es donde se aplican las dos reglas de servicio del sistema. El operador registra la llegada de una unidad al andén, anota cuántos pasajeros lleva y solicita el despacho. La aplicación calcula el porcentaje de ocupación a partir de la capacidad de la unidad y resuelve por sí sola qué hacer:

- Si la ocupación es menor al 25 %, retiene la unidad cinco minutos y muestra el tiempo restante en pantalla, actualizado cada segundo (RF-10).
- Si la ocupación supera el 150 %, marca el viaje como sobrecupo y genera una alerta crítica que llega a la bandeja del administrador (RF-09).

El operador no elige la estación: el sistema la toma de su cuenta, de modo que no puede registrar movimientos en una estación ajena. El andén se actualiza solo cada quince segundos.

Personal

El módulo de personal reúne en una sola pantalla, con tres pestañas, lo que antes estaba disperso: los expedientes de los pilotos con la vigencia de su licencia, el listado de guardias y los turnos que no alcanzan la cobertura mínima de su acceso. Las licencias vencidas y las próximas a vencer se distinguen con un distintivo de color, lo cual evita programar a un piloto con la licencia fuera de vigencia.

Cada piloto y cada guardia ven su propia información en los módulos *Mi expediente* y *Mi cobertura*. El servidor verifica la identidad contra el token de sesión, de modo que un piloto que intente consultar el listado general recibe un rechazo con código 403 aunque conozca la dirección del recurso.

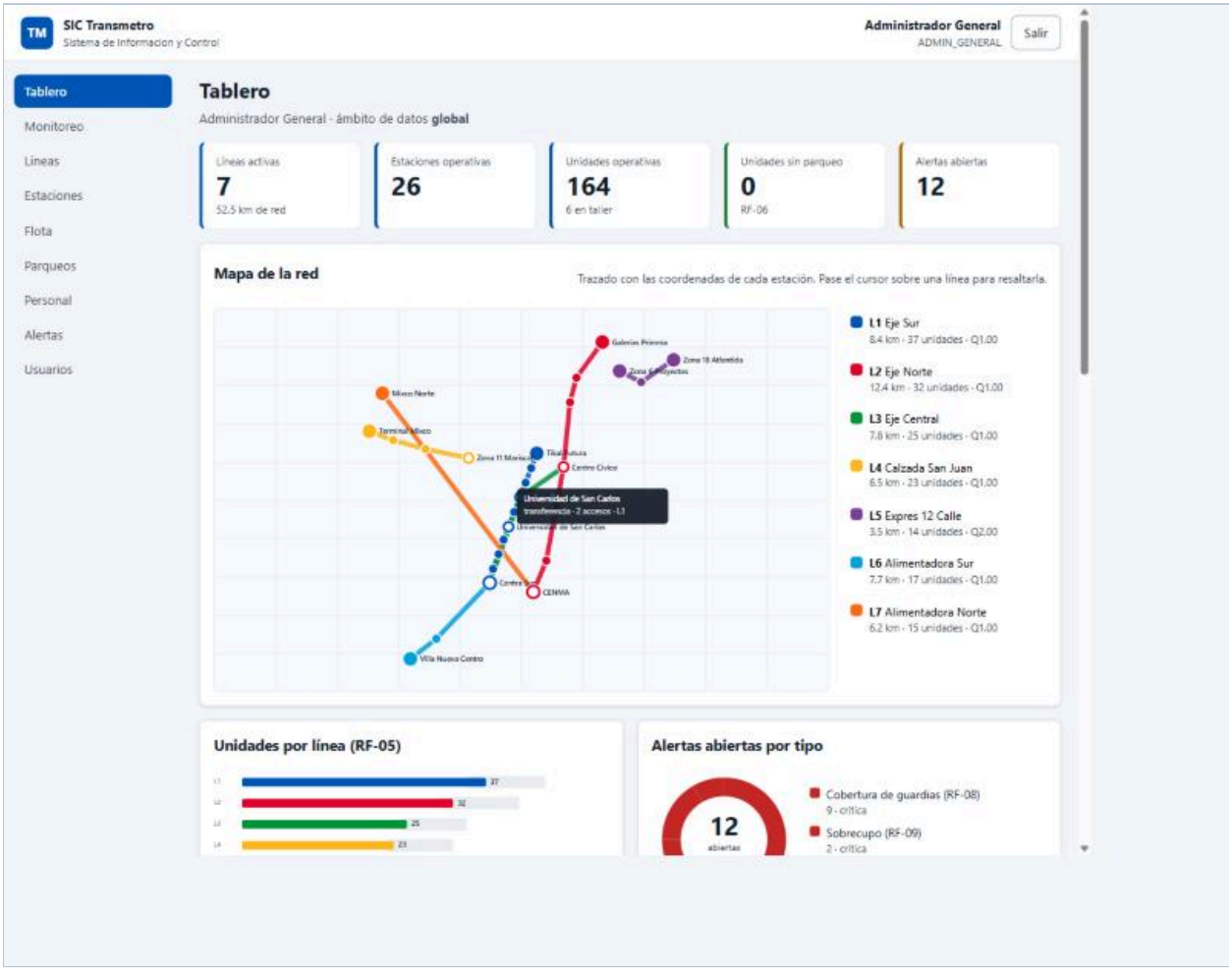
Tablero con Mapa e Indicadores

El tablero dejó de ser una tabla de líneas para convertirse en el panel de control del sistema. Presenta cinco indicadores, el mapa de la red y tres gráficas. Todo se dibuja en SVG dentro de la propia aplicación, sin librerías de gráficas ni servicios externos de mapas: el sistema debe seguir siendo útil en estaciones con conectividad intermitente, y un mapa que depende de internet deja de servir justo cuando más se necesita.

Pantallas del Sistema

Figura 23

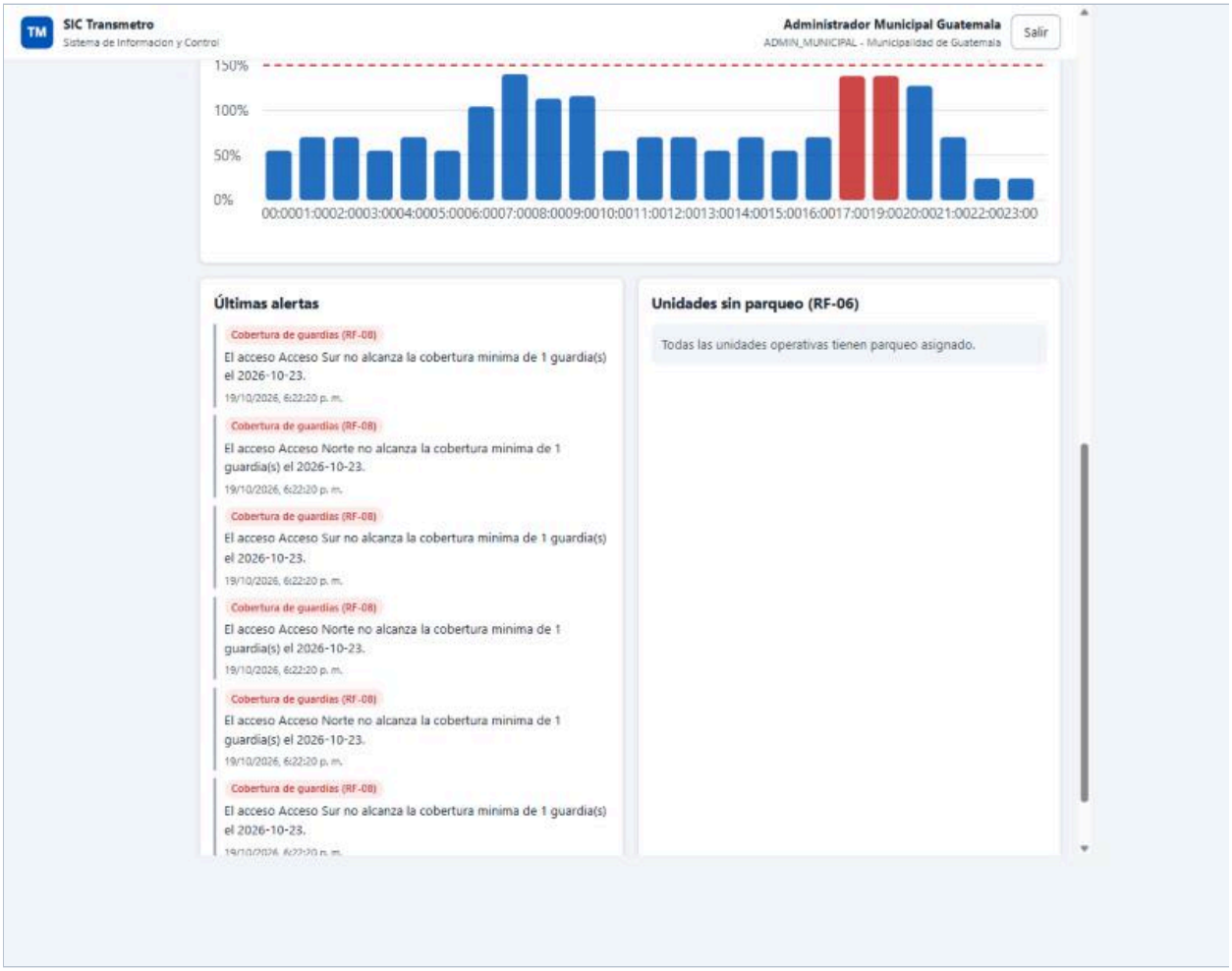
Tablero con el mapa de la red, los indicadores y las gráficas



Nota. El mapa se traza con las coordenadas de cada estación. Las terminales se dibujan con mayor tamaño y las estaciones de transferencia con círculo hueco; al pasar el cursor sobre una estación se muestran su tipo, sus accesos y las líneas que la sirven. La leyenda de la derecha indica la longitud, las unidades y la tarifa de cada línea.

Figura 24

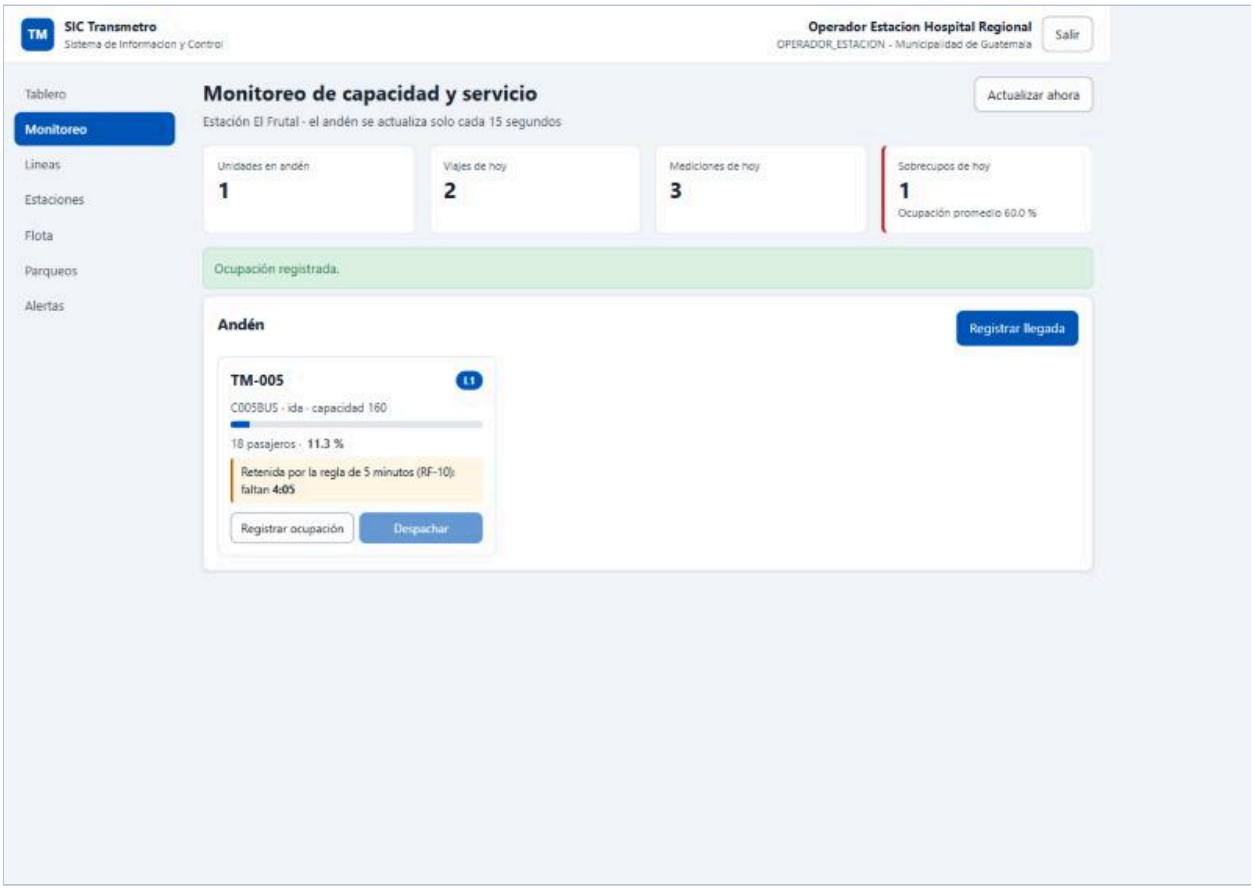
Ocupación promedio por hora en las últimas 24 horas



Nota. La línea punteada marca el umbral de sobrecupo del 150 % (RF-09) y las barras rojas señalan las horas en que se superó. La curva reproduce el patrón del servicio, con picos en la mañana y en la tarde. Debajo se muestran las últimas alertas y las unidades sin parqueo.

Figura 25

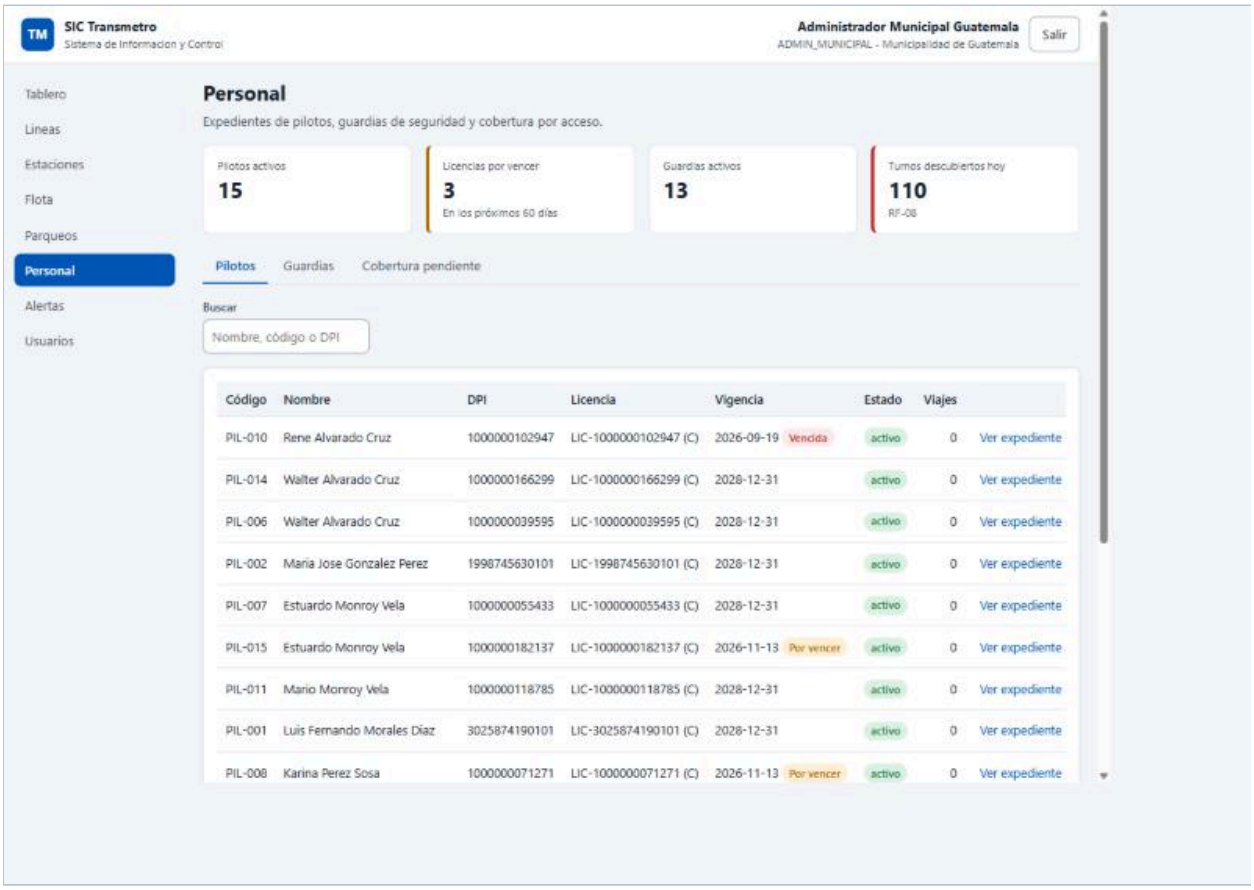
Andén del operador con la regla de espera aplicada en vivo



Nota. La unidad TM-005 lleva 18 pasajeros de 160 lugares, es decir 11.3 % de ocupación. Como no alcanza el 25 % mínimo, el sistema la retiene y muestra que faltan 4:05 para poder despacharla; el botón Despachar permanece deshabilitado mientras tanto (RF-10).

Figura 26

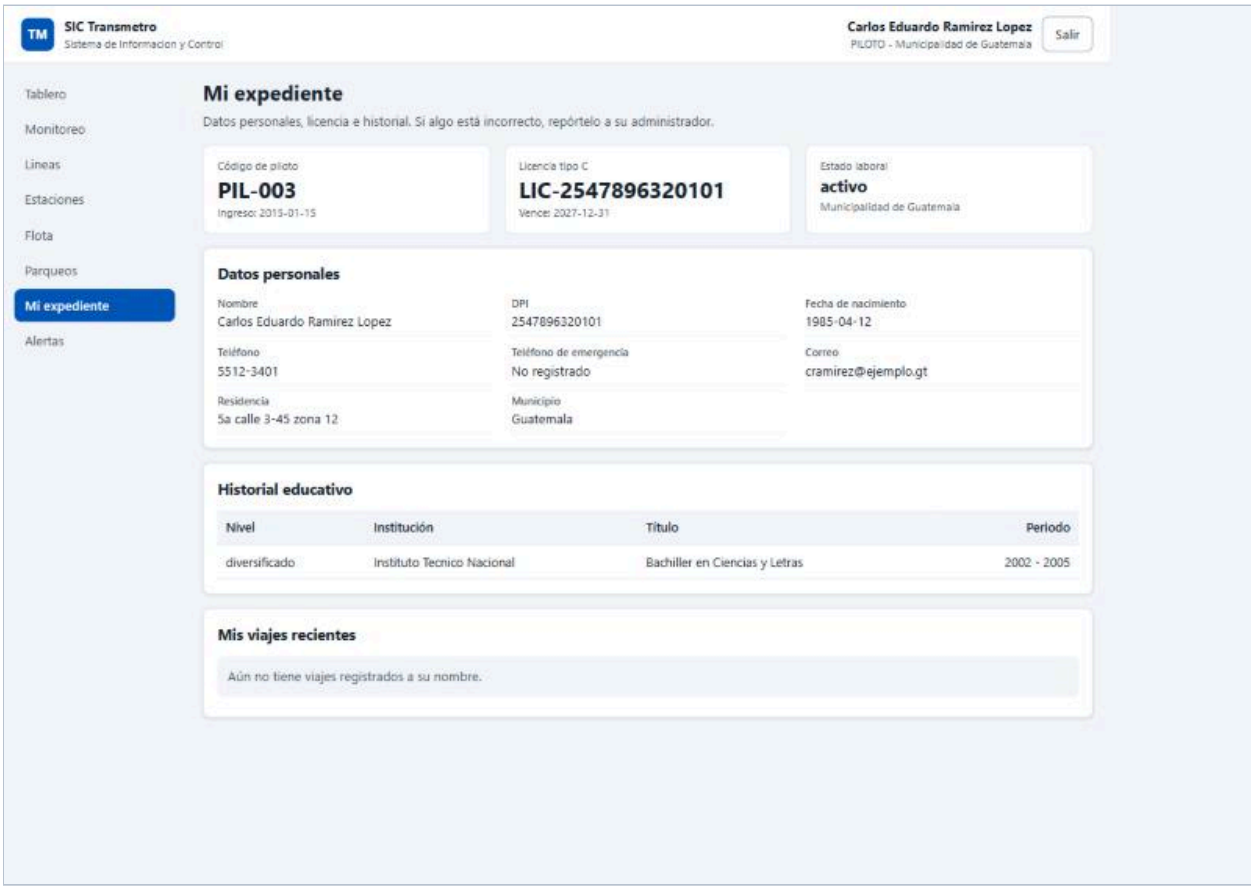
Módulo de personal con el control de licencias de los pilotos



Nota. Los indicadores superiores resumen 15 pilotos activos, 3 licencias por vencer en los próximos 60 días, 13 guardias activos y 110 turnos descubiertos en el día (RF-08). En el listado, la licencia del piloto PIL-010 aparece marcada como vencida.

Figura 27

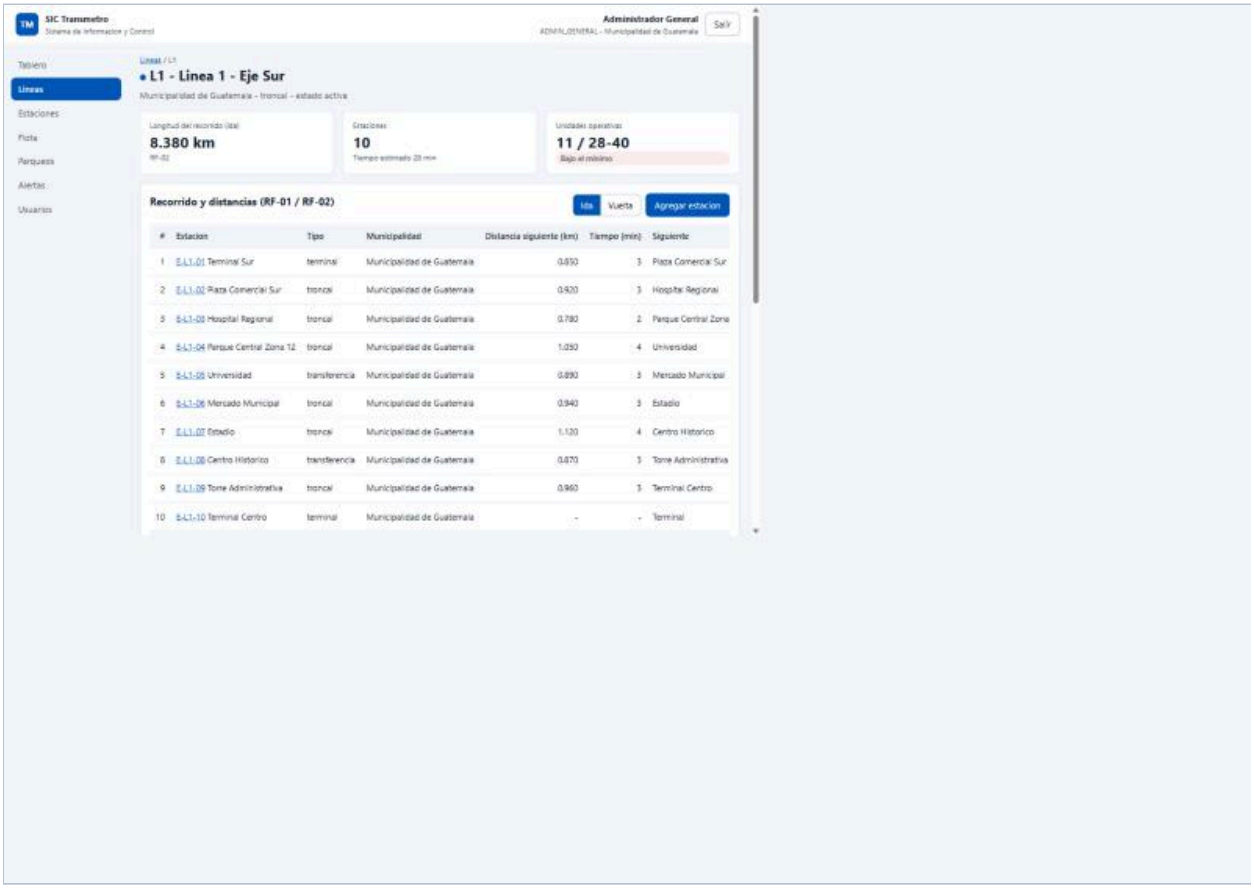
Expediente propio del piloto



Nota. El piloto accede únicamente a su expediente: código, licencia, estado laboral, datos personales, historial educativo y sus viajes recientes.

Figura 28

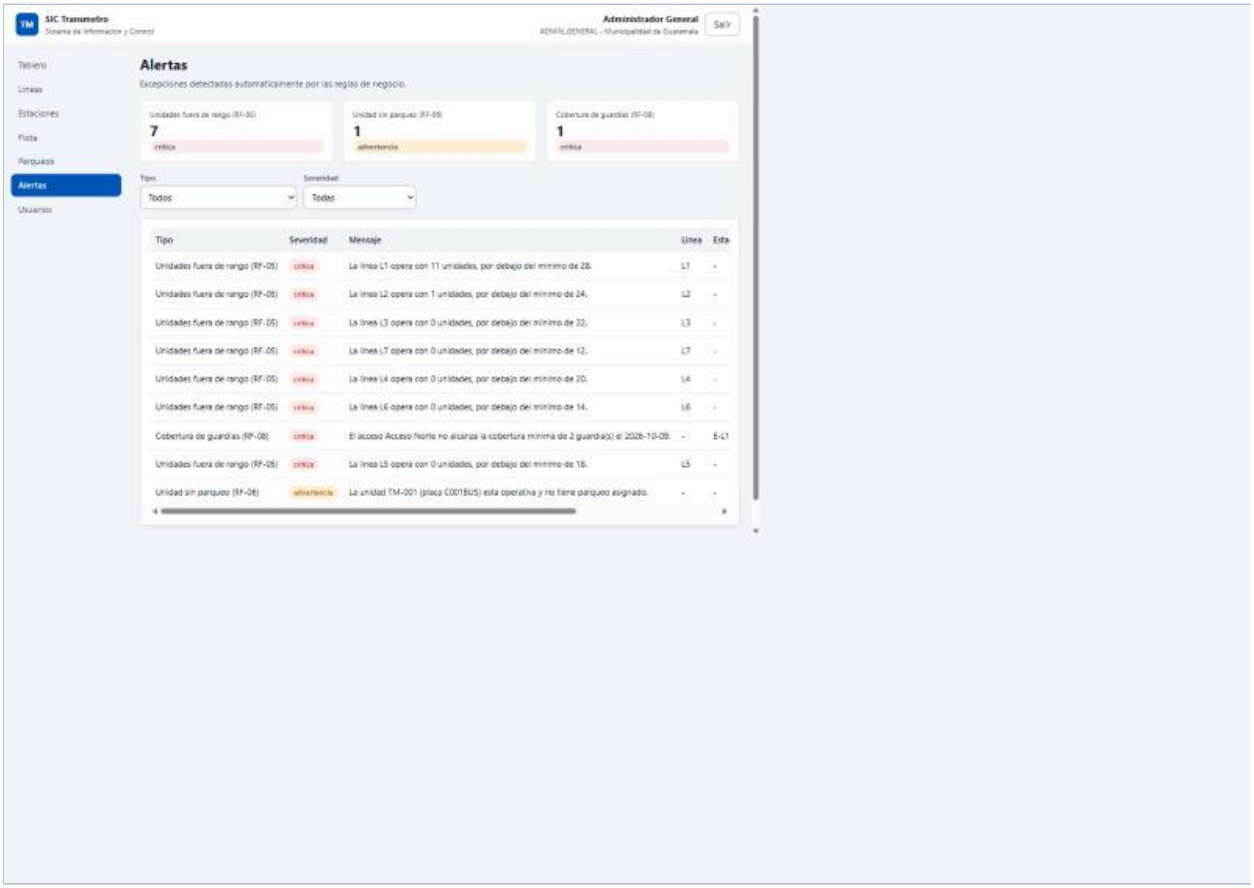
Detalle de una línea con su recorrido y las distancias entre estaciones



Nota. La longitud de la línea no está almacenada: resulta de sumar las distancias de los tramos del recorrido, de modo que al corregir un tramo el total se actualiza solo.

Figura 29

Bandeja de alertas generadas automáticamente por la base de datos



Nota. Las alertas no las crea ningún usuario: las generan los disparadores del motor al detectar líneas fuera de su rango de unidades, accesos sin la cobertura mínima de guardias, unidades sin parqueo y viajes con sobrecupo.

Comportamiento Verificado

Tabla 26

Flujos probados sobre el sistema en ejecución

Flujo	**Resultado observado**
Inicio de sesión	Ingreso correcto; el menú se arma con los módulos del rol y de su ámbito
Consulta del tablero	Siete líneas en el mapa, 52.5 km de red, 26 estaciones, 164 unidades operativas y 12 alertas abiertas
Recorrido de una línea	Estaciones en orden, con la distancia a la siguiente y la longitud total calculada
Reasignación de unidad	El sistema exige confirmación, cierra la asignación anterior y actualiza el conteo de ambas líneas en la misma operación
Registro de llegada al andén	La unidad entra al andén de la estación del operador y queda disponible para medir su ocupación
Registro de ocupación baja	Con 18 pasajeros de 160 el sistema calcula 11.3 % y activa la retención de cinco minutos
Intento de despacho anticipado	Rechazado, con el mensaje de la regla y el tiempo que falta; el botón queda deshabilitado
Registro de sobrecupo	Con 256 pasajeros de 160 el sistema calcula 160 % y genera alerta crítica
Consulta de expediente propio	El piloto ve sus datos; el acceso al listado general queda denegado
Consulta de turnos propios	El guardia ve sus turnos por acceso y puede registrar su asistencia
Unidades sin parqueo	Con las 164 unidades operativas asignadas a un espacio, el panel queda en cero

Nota. Las pruebas se ejecutaron sobre PostgreSQL 18.3 con los datos de ejemplo del guion de carga.

Inventario Técnico de lo Construido

Tabla 27

Tamaño del sistema al 19 de octubre de 2026

Elemento	Antes	Ahora
Módulos con interfaz	8	12
Pantallas del frontend	11	15
Puntos de acceso de la API	51	75
Tablas en la base de datos	32	33
Líneas de código	10 040	12 941

Nota. Las líneas de código corresponden a los archivos de código, hoja de estilos y fuentes de diagramas del repositorio, sin contar dependencias externas; es la misma medición que reporta el documento de métricas de software. La base de datos suma además 12 vistas, 15 funciones, 29 disparadores, 77 llaves foráneas, 49 restricciones de verificación y 3 restricciones de exclusión.

Estado del Avance

De los dieciséis requerimientos funcionales, doce están implementados y en funcionamiento: RF-01, RF-02, RF-03, RF-04, RF-05, RF-06, RF-07, RF-08, RF-09, RF-10, RF-13 y RF-15. El RF-16, la bitácora de auditoría, funciona de forma transversal en la base de datos y registra cada cambio, aunque todavía no tiene pantalla de consulta.

Quedan pendientes de interfaz tres requerimientos: RF-11 de administración técnica y conectividad, RF-12 de tarjetas y conciliación de pagos, y RF-14 de reportería gerencial. Sus tablas, reglas y vistas ya existen en la base de datos, de modo que el trabajo restante es la pantalla y los puntos de acceso correspondientes.

Parte V. Plan de Pruebas y Criterios de Aceptación

Objetivo y Alcance del Plan

Objetivo y Alcance

Este plan define cómo se verifica que el Sistema de Información y Control del Transmetro (SIC Transmetro) cumple sus requerimientos. El objetivo no es demostrar que el sistema funciona, sino buscar las condiciones en que falla, de modo que los defectos se encuentren antes de la entrega y no durante la operación.

Esta revisión incorpora los módulos construidos después de la primera ejecución del plan: monitoreo de capacidad y servicio, personal, mi expediente, mi cobertura y el tablero con mapa de la red e indicadores. Con ellos, los requerimientos RF-07, RF-08, RF-09 y RF-10 dejaron de estar cubiertos solo por la base de datos y pasaron a verificarse también desde la interfaz, mediante los casos CP-27 a CP-38.

Tabla 28

Alcance de las pruebas

Se prueba	No se prueba
Las siete reglas de negocio duras: RF-04, RF-05, RF-06, RF-08, RF-09, RF-10 y RF-16	El motor PostgreSQL, por tratarse de software de terceros con su propia verificación
La matriz de permisos completa: 5 roles por 8 módulos	Las bibliotecas de terceros declaradas en el stack
Los doce módulos construidos: autenticación, tablero, monitoreo, personal, mi expediente, mi cobertura, líneas, estaciones, flota, parqueos, alertas y usuarios	Los módulos de administración técnica, conciliación de pagos y reportería gerencial, aún sin interfaz
Los requerimientos no funcionales de seguridad, tiempo de respuesta y disponibilidad	La infraestructura del proveedor de nube

Nota. Los módulos excluidos tienen sus tablas y reglas creadas en la base de datos; sus pruebas se incorporarán conforme se construya su interfaz.

Estrategia de Pruebas

La estrategia se organiza en cinco niveles, del más barato al más caro de ejecutar. Un defecto encontrado en el nivel inferior cuesta una fracción de lo que cuesta encontrarlo en el superior (Pressman y Maxim, 2020).

Tabla 29

Niveles de prueba definidos para el proyecto

Nivel	**Qué verifica**	**Herramienta**	**Cuándo se ejecuta**
Restricciones de base de datos	Que las reglas duras rechacen los datos inválidos	Guiones SQL sobre PGlite	Con cada cambio al esquema
Unitarias	Funciones aisladas: niveles de permiso, traducción de errores, esquemas de validación	Vitest	Con cada cambio al código
Integración de la interfaz de programación	Un caso por punto de acceso y uno por código de error de negocio	Vitest con Supertest	Con cada cambio al servidor
Interfaz de usuario	Flujos por rol: ingreso, módulo permitido, acción denegada	Verificación manual guiada por guion	Al cierre de cada sprint
Carga	Comportamiento con cincuenta operadores concurrentes	k6	Sprint 7 y antes de la entrega

Nota. La base de la estrategia es deliberadamente la capa de datos, porque es donde residen las reglas de negocio de este sistema.

Ambiente de Pruebas

Tabla 30

Configuración del ambiente de pruebas

Elemento	Configuración
Motor de base de datos	PostgreSQL 18.3 mediante PGlite, ejecutado en el puerto 5433
Servidor de aplicación	Node.js 20 con la interfaz REST en el puerto 4000
Aplicación web	Vite en el puerto 5174
Datos de prueba	Los del guion de carga inicial: 5 municipalidades, 7 líneas, 26 estaciones, 39 accesos, 170 unidades, 15 pilotos, 14 guardias, 5 usuarios y 23 horas de historial de operación
Restablecimiento	El comando de reinicio borra los datos y vuelve a aplicar el guion completo, de modo que cada ciclo parte del mismo estado
Usuarios de prueba	Uno por rol, con la misma contraseña

Nota. El ambiente se levanta completo en cualquier equipo con Node.js, sin instalar un servidor de base de datos.

Casos de Prueba Ejecutados

Los casos siguientes se ejecutaron sobre el sistema en funcionamiento. La columna Resultado indica lo observado, no lo esperado.

Tabla 31

Casos de prueba de reglas de negocio

ID	RF	Caso	Resultado esperado	Resultado
CP-01	RF-04	Asignar a la línea L2 una unidad que ya está asignada a la L1	Rechazo con error de exclusión	Rechazado por ex_bus_una_linea_a_la_vez. Correcto
CP-02	RF-04	Reasignar la misma unidad confirmando el cierre de la asignación anterior	Se cierra la anterior y se crea la nueva	La unidad quedó en L2 y la anterior en el historial. Correcto
CP-03	RF-06	Asignar un segundo bus a un espacio de parqueo ya ocupado	Rechazo con error de exclusión	Rechazado por ex_espacio_un_bus. Correcto
CP-04	RF-06	Dejar una unidad operativa sin parqueo asignado	Aparece en la bandeja de excepciones	La unidad TM-001 apareció de inmediato. Correcto
CP-05	RF-10	Despachar una unidad con 20 % de ocupación a los 3 minutos de su llegada	Rechazo por restricción de verificación	Rechazado por ck_viaje_espera_minima. Correcto
CP-06	RF-10	Despachar la misma unidad a los 6 minutos	Aceptado	Aceptado. Correcto
CP-07	RF-09	Registrar 250 pasajeros en una unidad de capacidad 160	Marca de sobrecupo y alerta crítica	Sobrecupo verdadero, 156.25 %, alerta generada. Correcto
CP-08	RF-05	Dejar la línea L1 con 11 unidades frente a un mínimo de 28	Alerta crítica de rango	Alerta generada y visible en el tablero. Correcto
CP-09	RF-08	Acceso con mínimo de 2 guardias y cobertura incompleta	Alerta de cobertura con el detalle de turnos	Alerta generada con los turnos descubiertos. Correcto
CP-10	RF-16	Modificar el nombre de una línea	Registro en bitácora con usuario, acción y campos modificados	Registro creado con los tres datos. Correcto
CP-11	RF-02	Consultar la longitud de la línea L1	Suma de los nueve tramos del recorrido	8.380 km, coincide con la suma manual. Correcto
CP-12	RF-01	Ejecutar el guion completo dos veces seguidas	Se recrea el esquema sin errores	33 tablas y 12 vistas creadas en ambas ejecuciones. Correcto

Nota. Los casos CP-01 a CP-12 se ejecutaron de forma manual sobre PostgreSQL 18.3. Su automatización está programada para el Sprint 1.

Tabla 32

Casos de prueba automatizados de seguridad y permisos

ID	RF	Caso	Resultado
CP-13	RF-15	Acceder a una ruta protegida sin token	Rechazado con código 401. Correcto
CP-14	RF-15	Acceder con un token de firma inválida	Rechazado con código 401. Correcto
CP-15	RF-15	Enviar credenciales con formato inválido	Rechazado con código 400 antes de consultar la base. Correcto
CP-16	RF-15	Solicitar una ruta inexistente	Código 404 con el formato de error uniforme. Correcto
CP-17	RF-15	Leer líneas con nivel de lectura	Permitido. Correcto
CP-18	RF-15	Crear una línea teniendo solo nivel de lectura	Rechazado con código 403. Correcto
CP-19	RF-15	Acceder al módulo de flota sin tenerlo en la matriz	Rechazado con código 403. Correcto
CP-20	RF-15	Dar de baja una línea con nivel de escritura, sin administración	Rechazado con código 403. Correcto
CP-21	RF-05	Crear una línea con máximo menor que el mínimo	Rechazado con código 400 y detalle por campo. Correcto

Nota. Estos nueve casos están automatizados con Vitest y Supertest, y se ejecutan con un solo comando.

Tabla 33

Casos de prueba de interfaz

ID	Caso	Resultado
CP-22	Ingreso con el usuario administrador general	Acceso concedido; el menú muestra los módulos de su matriz de permisos. Correcto
CP-23	Consulta del tablero	Las siete líneas con unidades, rango, longitud y situación. Correcto
CP-24	Consulta del recorrido de una línea	Diez estaciones en orden con su distancia y la longitud total. Correcto
CP-25	Reasignación de unidad desde la pantalla de flota	Se solicita confirmación, se ejecuta y los conteos de ambas líneas se actualizan. Correcto
CP-26	Consulta de la bandeja de alertas	Nueve alertas generadas por el sistema, con tipo, severidad y mensaje. Correcto

Nota. Los casos de interfaz se ejecutaron de forma manual sobre el sistema en funcionamiento, en la versión del 9 de octubre de 2026.

Casos de los Módulos Incorporados

Los casos siguientes corresponden a los módulos construidos después de la primera ejecución del plan: monitoreo, personal, mi expediente, mi cobertura y el tablero con mapa e indicadores.

Tabla 34

Casos de prueba de los módulos nuevos

ID	RF	Caso	Resultado esperado	Resultado
CP-27	RF-09	Registrar la llegada de una unidad al andén desde la cuenta del operador	La unidad aparece en el andén de su estación	Unidad TM-005 en el andén de El Frutal. Correcto
CP-28	RF-10	Registrar 18 pasajeros en una unidad de 160 lugares	Ocupación de 11.3 % y retención por la regla de cinco minutos	Retenida, con cuenta regresiva visible en pantalla. Correcto
CP-29	RF-10	Pulsar Despachar antes de cumplir los cinco minutos	Acción bloqueada, con el tiempo restante a la vista	Botón deshabilitado y mensaje de la regla. Correcto
CP-30	RF-09	Registrar 256 pasajeros en una unidad de 160 lugares	Marca de sobrecupo, 160 % y alerta crítica	Sobrecupo verdadero y alerta crítica en la bandeja. Correcto
CP-31	RF-09	Consultar el andén con la cuenta del operador	Solo las unidades de su propia estación	Andén de El Frutal, sin unidades de otras estaciones. Correcto
CP-32	RF-07	Consultar el expediente propio con la cuenta de piloto	Datos, licencia, historial educativo y viajes del propio piloto	Expediente del piloto PIL-003 mostrado. Correcto
CP-33	RF-07	Solicitar el listado general de pilotos con la cuenta de piloto	Rechazo por permisos	Rechazado con código 403. Correcto
CP-34	RF-07	Consultar el listado de pilotos con la cuenta de administrador	Licencias vencidas y próximas a vencer distinguidas	15 pilotos activos, la licencia de PIL-010 marcada como vencida y tres por vencer en el indicador. Correcto
CP-35	RF-08	Consultar los turnos propios con la cuenta de guardia	Solo los turnos del guardia que ingresó	Turnos propios por acceso, con opción de registrar asistencia. Correcto
CP-36	RF-08	Consultar la cobertura pendiente con la cuenta de administrador	Turnos descubiertos del día con su acceso y estación	110 turnos descubiertos listados. Correcto
CP-37	RF-02	Consultar el mapa de la red en el tablero	Las siete líneas trazadas con sus estaciones en orden	Mapa dibujado con 26 estaciones y 52.5 km de red. Correcto
CP-38	RF-09	Consultar la gráfica de ocupación por hora	Barras por hora con el umbral de sobrecupo marcado	23 horas graficadas; las horas con sobrecupo en rojo. Correcto

Nota. Los casos CP-27 a CP-38 se ejecutaron de forma manual sobre el sistema en funcionamiento el 19 de octubre de 2026, con las cuentas de operador, piloto, guardia y administrador.

Casos de Prueba Pendientes

Tabla 35

Casos programados para sprints posteriores

ID	RF o RNF	Caso	Sprint
CP-39	RF-04	Dos transacciones concurrentes asignando la misma unidad; una debe fallar	3
CP-40	RF-07	Alta de expediente de piloto con historial educativo desde la interfaz	4
CP-41	RF-11	Registro de estado de equipo y conectividad por estación	6
CP-42	RF-12	Reenvío de una transacción con la misma referencia externa; no debe duplicarse	6
CP-43	RF-14	Exportación de reportes a PDF y Excel con marca de clasificación	7
CP-44	RNF	Tiempo de respuesta menor a 3 segundos con 50 usuarios concurrentes	7
CP-45	RNF	Restauración completa desde respaldo, con tiempo medido	7

Nota. CP-39 requiere ejecutar dos transacciones simultáneas, lo cual no es posible con PGlite por atender una conexión a la vez. Se ejecutará contra una instancia administrada de PostgreSQL. El módulo de personal ya permite consultar los expedientes; CP-40 cubre el alta desde la interfaz, que todavía se realiza por guion.

Criterios de Entrada, Salida y Suspensión

Tabla 36

Criterios del proceso de pruebas

Criterio	Condición
Entrada	El código compila, el guion de base de datos se aplica sin errores y el ambiente está levantado
Salida	Todos los casos de severidad alta aprobados, ningún defecto crítico abierto y la cobertura de servicios igual o mayor al 70 %
Suspensión	Un defecto crítico impide ejecutar más del 30 % de los casos planificados
Reanudación	El defecto crítico está corregido y verificado, y los casos previamente aprobados se vuelven a ejecutar

Nota. Elaboración propia.

Gestión de Defectos

Tabla 37

Clasificación de defectos por severidad

Severidad	Definición	Tiempo de atención
Crítica	Impide usar el sistema o permite violar una regla de negocio dura	Inmediato; detiene el sprint
Alta	Un módulo no cumple su requerimiento, sin alternativa de uso	Dentro del mismo sprint
Media	Un módulo falla pero existe otra forma de lograr el resultado	Sprint siguiente
Baja	Problema estético o de redacción	Según disponibilidad

Nota. Todo defecto hallado en producción obliga a crear primero un caso de prueba que lo reproduzca, y solo después a corregirlo.

Tabla 38

Defectos registrados hasta la fecha

ID	Descripción	Severidad	Estado
DEF-01	El guion de carga inicial no evaluaba la alerta de rango en las líneas sin unidades asignadas	Media	Corregido y verificado
DEF-02	Etiqueta superpuesta en el diagrama de estados de la unidad	Baja	Corregido
DEF-03	El esquema de búsqueda no se aplicaba a la conexión, por lo que toda consulta fallaba	Crítica	Corregido y verificado
DEF-04	El conjunto de conexiones abría varias en paralelo contra un motor que atiende una	Alta	Corregido y verificado

Nota. Los defectos DEF-03 y DEF-04 se detectaron al conectar el sistema a una base de datos real por primera vez, lo cual confirma el valor de probar la integración temprano.

Criterios de Aceptación

Los criterios siguientes son las condiciones que el sistema debe cumplir para que la entrega sea aceptada. Cada uno es verificable: se redacta de forma que la respuesta solo pueda ser cumple o no cumple.

Criterios por Requerimiento Funcional

Tabla 39

Criterios de aceptación de los requerimientos implementados

RF	Criterio de aceptación	Estado
RF-01	El sistema permite registrar líneas y estaciones, y asociar estaciones a una línea indicando su orden de recorrido. Una estación no puede repetirse en el mismo sentido de una línea	Cumple
RF-02	El sistema calcula la longitud de cada línea como la suma de las distancias entre sus estaciones, y el valor cambia al corregir cualquier tramo	Cumple
RF-03	El sistema registra los accesos de cada estación, con su tipo, cantidad de torniquetes y mínimo de guardias requerido	Cumple
RF-04	El sistema impide que una unidad quede asignada a dos líneas en periodos que se traslapen, incluso ante operaciones simultáneas	Cumple
RF-05	El sistema genera una alerta cuando una línea activa opera fuera de su rango de unidades, y la cierra al normalizarse	Cumple
RF-06	El sistema impide que dos unidades ocupen el mismo espacio de parqueo y lista las unidades operativas sin parqueo asignado	Cumple
RF-07	El sistema mantiene el expediente de cada piloto con sus datos de contacto, residencia y su historial educativo, y cada piloto consulta el suyo sin ver el de los demás	Cumple
RF-08	El sistema valida la cobertura mínima de guardias por acceso y turno, alerta cuando no se alcanza y permite al guardia registrar su asistencia	Cumple
RF-09	El sistema registra la ocupación por unidad, marca el sobrecupo al superar el 150 % de la capacidad y lo presenta por hora en el tablero	Cumple
RF-10	El sistema impide despachar una unidad con menos del 25 % de ocupación antes de cinco minutos, y muestra al operador el tiempo restante	Cumple
RF-13	Cada línea y estación está asociada a una municipalidad responsable	Cumple
RF-15	Cada rol accede únicamente a los módulos de su matriz de permisos, y dentro del ámbito de datos que le corresponde	Cumple
RF-16	Toda modificación a las tablas críticas queda registrada con usuario, fecha, acción y valores anteriores y nuevos	Cumple

Nota. Los requerimientos anteriores fueron verificados mediante los casos CP-01 a CP-38. Los criterios de RF-07 a RF-10 pasaron de "base cumple" a "cumple" al construirse los módulos de monitoreo, personal, mi expediente y mi cobertura.

Tabla 40

Criterios de aceptación de los requerimientos pendientes

RF	Criterio de aceptación	Estado
RF-11	El sistema registra el estado del equipo de cómputo y la conectividad de cada estación	Base cumple; falta interfaz
RF-12	El sistema concilia los pagos por tarjeta por estación y día, y señala las diferencias	Base cumple; falta interfaz
RF-14	El sistema genera reportes consolidados exportables a PDF y Excel	Pendiente

Nota. "Base cumple" significa que las tablas y las reglas están implementadas y verificadas en el motor de base de datos, pero el módulo de interfaz correspondiente no ha sido construido.

Criterios por Requerimiento No Funcional

Tabla 41

Criterios de aceptación de los requerimientos no funcionales

Requerimiento	Criterio medible	Estado
Disponibilidad	Igual o mayor al 99 % de los minutos del horario operativo, medido durante un mes	Pendiente de medición
Tiempo de respuesta	El percentil 95 de los diez puntos de acceso más usados es menor a 3 segundos con 50 usuarios concurrentes	Pendiente de medición
Seguridad	Las contraseñas se almacenan con bcrypt, la comunicación viaja por HTTPS y la cuenta se bloquea tras cinco intentos fallidos	Cumple
Usabilidad	Un operador ejecuta el registro de ocupación desde un dispositivo móvil sin capacitación previa, en menos de un minuto	Pendiente de prueba con usuarios
Escalabilidad	Agregar una municipalidad, línea o estación no requiere modificar el código ni el esquema	Cumple
Capas desacopladas	La aplicación web, la interfaz REST y la base de datos se despliegan por separado	Cumple
Clasificación de la información	Cada tabla tiene asignada su clasificación conforme al Decreto 57-2008	Cumple
Respaldo y recuperación	Existe respaldo automático diario y una restauración probada con tiempo documentado	Pendiente de prueba

Nota. Los criterios pendientes corresponden al Sprint 7, conforme al plan de desarrollo.

Criterios Globales de Aceptación

La entrega se considera aceptada cuando se cumplen simultáneamente las condiciones siguientes:

1. Los requerimientos funcionales comprometidos para la entrega están implementados y sus casos de prueba aprobados.
2. No existe ningún defecto abierto de severidad crítica o alta.
3. Las siete reglas de negocio duras están garantizadas por el motor de base de datos y verificadas con datos inválidos.

4. La matriz de permisos se comporta conforme a la especificación para los cinco roles.
5. El sistema se instala y se levanta siguiendo la documentación entregada, sin asistencia del desarrollador.
6. La documentación entregada corresponde a la versión del sistema entregada.

Anexo A. Carta de Aceptación de Entrega

Guatemala, _____ de _____ de 20____

Señores

Dirección del Transmetro

Municipalidad de Guatemala

Presente

Por este medio se hace constar que el día de hoy fue recibida y revisada la entrega del Sistema de Información y Control del Transmetro (SIC Transmetro), desarrollado por Luis Orozco, carné 3590 22 5339, como proyecto final del curso de Aseguramiento de la Calidad del Software.

El sistema centraliza en una sola base de datos la información del servicio, hoy dispersa entre municipalidades y operadores, y automatiza en el motor de base de datos las reglas de operación que antes dependían de verificación manual.

Tabla 42

Entregables recibidos

Entregable	**Contenido**
Modelo de datos	Diagrama entidad relación de 33 tablas, completo y por dominio
Script de base de datos	Archivo único ejecutable, verificado sobre PostgreSQL 18.3
Diagramas UML	15 diagramas: casos de uso, clases, secuencia, actividades, estados, componentes y despliegue
Código fuente	Interfaz REST en Node.js y aplicación web en React, publicados en el repositorio
Documentación técnica	Arquitectura, stack tecnológico, lenguajes y motor de base de datos
Documentación funcional	Descripción de los doce módulos construidos y guía de puesta en marcha
Plan de pruebas	Estrategia, 38 casos ejecutados y criterios de aceptación
Plan de métricas	Métricas de producto y de proceso con sus metas y umbrales

Nota. Todos los entregables se encuentran en el repositorio https://github.com/LuisO01/transmetro-sic

Verificación realizada: se comprobó que el sistema cumple los criterios de aceptación acordados, conforme al detalle siguiente:

1. Los requerimientos funcionales RF-01, RF-02, RF-03, RF-04, RF-05, RF-06, RF-07, RF-08, RF-09, RF-10, RF-13, RF-15 y RF-16 se encuentran implementados y en funcionamiento, con su interfaz de usuario construida y verificada.
2. Las siete reglas de negocio críticas están garantizadas por el motor de base de datos y fueron verificadas con datos inválidos, que el sistema rechazó.
3. La matriz de permisos opera conforme a lo especificado para los cinco roles definidos.
4. El sistema se instala y se pone en marcha siguiendo la documentación entregada.
5. No existen defectos abiertos de severidad crítica ni alta.

Alcance pendiente, conocido y aceptado: los módulos de administración técnica, conciliación de pagos y reportería gerencial, correspondientes a los requerimientos RF-11, RF-12 y RF-14, cuentan con su modelo de datos y sus reglas implementadas en la base, y quedan programados para las fases siguientes conforme al plan de desarrollo. Se deja constancia, además, de que los casos de prueba de los módulos de monitoreo y personal se ejecutaron de forma manual y su automatización queda programada para el sprint siguiente.

Observaciones:

_____

_____

En virtud de lo anterior, se da por aceptada la entrega descrita, y se firma la presente para los usos que al interesado convengan.

Tabla 43

Firmas

Desarrollador	**Catedrático del curso**	**Entidad receptora**
Luis Orozco, carné 3590 22 5339	Aseguramiento de la Calidad del Software	Nombre, cargo y sello
_____	_____	_____

Nota. La versión firmable de esta carta, en archivo independiente, es SIC_Transmetro_Carta_de_Aceptacion.pdf.

Referencias

Congreso de la República de Guatemala. (2008). *Decreto 57-2008, Ley de Acceso a la Información Pública*. Diario de Centro América.

IEEE. (2008). *IEEE Standard for Software and System Test Documentation* (IEEE Std 829-2008). Institute of Electrical and Electronics Engineers.

ISO/IEC. (2011). *ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.

Meta Platforms. (s.f.). *React documentation*. https://react.dev

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. https://www.omg.org/spec/UML/

OpenJS Foundation. (s.f.). *Node.js documentation*. https://nodejs.org/docs

PostgreSQL Global Development Group. (s.f.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Pressman, R. S., y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.ª ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson Education.